

FIRAT ÜNİVERSİTESİ

MESLEK YÜKSEK OKULU TEKNİK BİLİMLER
Bilgisayar Teknolojileri Bölümü

SİSTEM ANALİZİ VE TASARIMI
Dersi Proje Uygulaması ve Dokümantasyonu

FIRAT PTS
PLAKA TANIMA SİSTEMİ

Geliştiren
240130012 – RÜYA METE

Proje Yürütücüleri
Öğr. Üyesi Merve PARLAK BAYDOĞAN

ÖNSÖZ VE TEŞEKKÜR

Günümüz dünyasında güvenlik ve otonom sistemler, kurumların dijital dönüşüm süreçlerinde en kritik rolleri üstlenmektedir. Fırat Üniversitesi kampüs nizamiyelerindeki araç giriş-çıkış süreçlerini modernize etmek, insan kaynaklı hataları en aza indirmek ve güvenliği maksimize etmek amacıyla geliştirdiğim "Fırat PTS (Plaka Tanıma Sistemi)" projesi, akademik hayatımda edindiğim teorik bilgileri pratiğe dönüştürdüğüm en kapsamlı çalışmalardan biri olmuştur.

Hayatım boyunca ve bu zorlu çalışma süresince manevi desteklerini hiçbir zaman esirgemeyen kıymetli aileme ve arkadaşlarıma sonsuz teşekkürü bir borç bilirim. Sistem Analizi ve Tasarımı dersi kapsamında ufkumu açan ve projeye yön vermemi sağlayan değerli proje yürütücüm Öğr. Üyesi Merve PARLAK BAYDOĞAN' a katkılarından dolayı minnettarım. Teşekkür ederim.

Rüya METE

Proje Maliyet Kesim Raporu

Proje adı: **Fırat PTS-Plaka Tanıma Sistemi**

Teknik Karmaşıklık Sorusu	Puan
1. Uygulama, güvenilir yedekleme ve kurtarma gerektiriyor mu?	4
2. Veri iletişimi gerekiyor mu?	5
3. Dağıtık işlem işlevleri var mı?	5
4. Performans kritik mi?	5
5. Sistem mevcut ve ağır yükü olan bir işletim ortamında mı çalışacak?	3
6. Sistem, çevrim içi veri girişi gerektiriyor mu?	5
7. Çevrim içi veri girişi, bir ara işlem için birden çok ekran gerektiriyor mu?	4
8. Ana kütükler çevrim-içi olarak mı günleniyor?	4
9. Girdiler, çıktılar, kütükler ya da sorgular karmaşık mı?	4
10. İçsel işlemler karmaşık mı?	5
11. Tasarlanacak kod, yeniden kullanılabilir mi olacak?	4
12. Dönüştürme ve kurulum, tasarımda dikkate alınacak mı?	2
13. Sistem birden çok yerde yerleşik farklı kurumlar için mi geliştiriliyor?	4
14. Tasarlanan uygulama, kolay kullanılabilir ve kullanıcı tarafından kolayca değiştirilebilir mi olacak?	3
Toplam (TKF)	57

Tablo 1: Teknik Karmaşıklık Faktörü Değerlendirme Tablosu

0: Hiçbir Etkisi Yok
1: Çok Az etkisi var
2: Etkisi Var
3: Ortalama Etkisi Var
4: Önemli Etkisi Var
5: Mutlaka Olmalı, Kaçınılamaz

Nihai Maliyet ve Satır Kestirimi Hesaplamaları:

- İşlev Noktası (İN) Formülü: AİN x [0.65 + (0.01 x TKF)]
- Hesaplama: 74 x [0.65 + (0.01 x 57)]
- Nihai İN Değeri: 74 x 1.22 = 90.28
- Tahmini Kaynak Kodu Satır Sayısı (LOC): İN x 30
- Toplam Tahmini Satır: 90.28 x 30 = 2708 LOC (Lines of Code)

1.GİRİŞ

1.1. Projenin Amacı ve Gerekçesi

1.2. Projenin Kapsamı

1.3. Literatür Taraması ve Teknolojik Arka Plan

1.3.1. Plaka Tespiti ve Görüntü İşleme Teknikleri

1.3.2. Optik Karakter Tanıma (OCR) Mimarisi

1.3.3. Dağıtık Mimari ve API Tabanlı Sistemler

1.4. Tanımlamalar ve Kısaltmalar

1.5. Kullanılacak Teknolojiler

2.PROJE PLANI VE YÖNETİMİ

2.1. Giriş

2.2. Projenin Plan Kapsamı ve Kısıtları

2.3. Proje Zaman-İş Planı (Gantt Şeması)

2.4. Proje Ekip Yapısı ve Rol Dağılımı (Full-Stack Yaklaşım)

2.5. Yazılım Maliyet Kestirimi (İşlev Noktası ve COCOMO Analizi)

2.5.1. İşlev Noktası (AİN) Hesaplaması

2.5.2. Teknik Karmaşıklık Faktörü (TKF) ve Satır Kestirimi

2.6. Önerilen Sistemin Teknik Tanımları

2.6.1. Minimum Sistem ve Donanım Gereksinimleri

2.7. Kullanılan Geliştirme Araçları (Ortamlar)

2.8. Proje Standartları ve Yaşam Döngüsü Modeli

2.9. Kalite, Konfigürasyon ve Risk Yönetim Planları

2.10. Veri Akış Diyagramı

3.GEREKSİNİM ANALİZİ

3.1. Mevcut Sistem İncelemesi

3.1.1. Mevcut Örgüt Yapısı ve İşlevsel Darboğazlar

3.1.2. Mevcut Veri Modeli

3.1.3. Var Olan Sistemin Değerlendirilmesi ve Eksiklikleri

3.2. Gereksinilen Sistemin Mantıksal Modeli

3.2.1. Sistem Aktörleri ve Roller

3.2.2. Kullanım Senaryoları
3.2.3. Genel Sistem Akışı
3.2.4. Bilgi Sistemleri ve Nesneler Arası İletişim
3.2.5. Başarım (Performans) Gerekleri
3.3. Arayüz (Modül) Gerekleri
3.3.1. Mobil Kullanıcı Arayüzü (UI/UX) Gereksinimleri
3.3.2. Donanım Arayüzü (Kamera, Ses ve Titreşim Motorları)
3.3.3. API ve İletişim Arayüzü

4.SİSTEM TASARIMI VE MİMARİSİ

4.1. Genel Tasarım ve Sistem Mimarisi
4.1.1. Uç Bilişim (Edge Computing) Yaklaşımı
4.1.2. İstemci - Sunucu (Client-Server) Mimarisi
4.2. Veritabanı ve Veri Modeli Tasarımı
4.2.1. SQLite Veritabanı Seçim Gerekçeleri
4.2.2. Tablo Tanımları
4.2.3. Varlık-İlişki (E-R) Diyagramı ve Şemalar
4.2.4. Veri Sözlüğü (Data Dictionary)
4.3. Dosya Sistemi ve Veri Depolama Stratejisi
4.3.1. Görüntülerin Veritabanı Yerine Disk Üzerinde Saklanması
4.3.2. Base64 Encoding ile Görüntü Taşıma İşlemleri
4.3.3. Sunucu Dizin (Directory) Yapısı ve İsimlendirme Standartları
4.3.4. İşletim Sistemi Dosya Yapıları ve PTS Sunucusu Uyumu
4.3.5. Kaynak Kod Dizin (Klasör) Hiyerarşisi
4.4. Süreç Tasarımı ve Uygulama Modülleri
4.4.1. Modüller Arası İlişkiler
4.4.2. Plaka Okuma ve Doğrulama Modülü
4.4.3. Sistem Yönetim Modülleri
4.4.4. Raporlama Modülü

5.SİSTEM GERÇEKLEŞTİRİMİ

5.1. Giriş

5.2. Yazılım Geliştirme Ortamları ve Diller

5.2.1. Programlama Dilleri ve Teknolojiler

5.2.2. Veritabanı Yönetim Sistemleri (VTYS) ve SQLite Mimarisi

5.3. Kodlama Stili ve Yapısal Standartlar

5.3.1. Açıklama Satırları ve Kod Biçimlemesi

5.3.2. Anlamlı İsimlendirme

5.4. Program Karmaşıklığı

5.4.1. McCabe Karmaşıklık Ölçütü Hesaplama

5.5. Olağan Dışı Durum Çözümleme

5.5.1. Olağandışı Durum Tanımları ve Yönetimi

6.DOĞRULAMA VE GEÇERLEME

6.1. Giriş

6.2. Sınama Yöntemleri (Kara Kutu ve Beyaz Kutu Testleri)

6.3. Birim ve Tümleştirme Testleri

6.4. Sistem Test Senaryoları Matrisi

6.4.1. Gündüz ve İdeal Koşullar Testi

6.4.2. Yasaklı Araç (Kırmızı Alarm) ve İhlal Tespiti Testi

6.4.3. Kayıtsız (Yabancı) Araç Tespiti Testi

6.4.4. Peş Peşe Geçiş Engelleme Testi

6.4.5. Çevrimdışı Kalma ve Ağ Kopması Testi

6.5. Kalite Güvence ve Sistem Onay Matrisi

7.BAKIM, KURULUM VE SÜRDÜRÜLEBİLİRLİK

7.1. Giriş

7.2. Sistemin Canlı Ortama Kurulumu

7.3. Yazılım Bakımı ve Optimizasyon

7.3.1. Biriken Görsel Logların Arşivlenmesi ve Silinmesi

7.3.2. SQLite Veritabanı Bakımı

8.PROJEDE ÖNGÖRÜLEN EKSİKLİKLER VE GELECEK ÇALIŞMALAR

8.1. Mevcut Sürümde Eksik Kalan Modüller ve Teknik Gerekçeleri

8.2. Projeye İleride Eklenebilecek Özellikler

8.3. Projenin Geliştirme Süresi Boyunca Bize Katkısı

9.PROJE TESLİM KILAVUZU

9.1. Python Sunucusunun Ayağa Kaldırılması

9.2. Ngrok Bağlantısının Sağlanması ve API Adreslerinin Güncellenmesi

9.3. Android Uygulamasının Derlenip (APK) Cihaza Yüklenmesi

10.SONUÇ VE DEĞERLENDİRME

10.1. Projenin Genel Değerlendirmesi (Artıları, Eksileri ve Tercih Sebepleri)

10.2. Projenin Geliştiriciye Akademik ve Mesleki Katkıları

11.KAYNAKLAR

ŞEKİLLER TABLOSU

Şekil 1: Veri Akış Diyagramı (DFD - 0. Seviye Bağlam Diyagramı) 19

Şekil 2: Kullanım Senaryoları (Use-Case) Diyagramı 22

Şekil 3: Genel Sistem Akış Diyagramı 24

Şekil 4: Mobil Kullanıcı Arayüzü Ekran Görüntüleri 26

Şekil 5: Donanım Arayüzü ve Alarm Ekranı 27

Şekil 6: KULLANICILAR Tablosunun Veri Yapısı ve Sütunları 29

Şekil 7: ARACLAR Tablosunun Fiziksel Görünümü 29

Şekil 8: Geçiş Kayıtları Tablosunun Fiziksel Görünümü 30

Şekil 9: Modüller Arası İlişkiler UML Sınıf Diyagramı 33

Şekil 10: Plaka Okuma ve Doğrulama Modülü 34

Şekil 11: Sistem Yönetim Modülleri 35

Şekil 12: PDF Raporlama Modülü Ekran Görüntüsü 36

Şekil 13: Gündüz ve İdeal Koşullar Test Sonucu Ekranı 37

Şekil 14: Python Sunucusunun Başlatılması ve Veritabanı İlklendirme Akışı 37

1.GİRİŞ

1.1. Projenin Amacı ve Gerekçesi

Günümüzde teknolojik gelişmelerin hız kazanması, kurumların ve geniş kampüs alanlarının güvenlik süreçlerini doğrudan etkilemektedir. Akıllı ulaşım ve güvenlik sistemleri (AUS), insan faktöründen kaynaklanan hataları minimize etmeyi, işlem hızını artırmayı ve geçmişe dönük verilerin dijital ortamda güvenle saklanmasını hedeflemektedir.

Bu bağlamda, araç giriş-çıkışlarının yoğun olduğu üniversite kampüsleri gibi alanlarda geleneksel yöntemlerle (fiziksel defter tutma, gözle kimlik kontrolü vb.) güvenliği sağlamak hem ciddi zaman kayıplarına yol açmakta hem de güvenlik açıklarına zemin hazırlamaktadır.

Projenin Gerekçesi: Fırat Üniversitesi nizamiyelerinde mevcut durumda uygulanan manuel araç kontrol sistemleri, özellikle sabah mesai başlangıçlarında ve akşam çıkış saatlerinde ciddi bir trafik darboğazı yaratmaktadır.

Güvenlik görevlilerinin her aracı durdurması, kurum kimliği sorması ve ziyaretçi/misafir araçların plakalarını fiziksel defterlere kaydetmesi araç başına ortalama 15-30 saniye kayba neden olmaktadır. Ayrıca, manuel tutulan kayıtlar olası bir adli veya idari soruşturma durumunda geriye dönük aramaları neredeyse imkansız hale getirmektedir.

Fiziksel kayıt defterleri kaybolma, yıpranma veya yetkisiz kişilerin eline geçme riski taşıdığından Kişisel Verilerin Korunması Kanunu (KVKK) açısından da riskler barındırmaktadır. Piyasada bulunan endüstriyel Otomatik Plaka Tanıma (ANPR) kamera sistemleri ise çok yüksek kurulum, lisans ve bakım maliyetleri gerektirdiği için her nizamiye noktasına entegre edilmesi kurum bütçesini zorlamaktadır.

Projenin Amacı: Fırat PTS projesinin temel amacı; yüksek maliyetli endüstriyel ANPR sistemlerine alternatif olarak, halihazırda güvenlik görevlilerinin erişebileceği standart Android işletim sistemli akıllı telefonları "mobil bir güvenlik terminaline" dönüştürmektir. Proje, araç plakalarını cihaz üzerinde çalışan makine öğrenmesi algoritmalarıyla saniyeler içinde tespit etmeyi, okunan plakayı yerel sunucuda sorgulayarak aracın kampüse giriş yetkisini anlık olarak doğrulamayı amaçlar. Böylece donanım maliyetleri minimuma indirilirken, yazılımın otonom karar destek mekanizmaları sayesinde kampüs güvenliği maksimize edilecektir.

1.2. Projenin Kapsamı

Bu proje, yazılım mühendisliği disiplinlerine uygun olarak geliştirilmiş tam yığın (Full-Stack) bir mobil otomasyon sistemini kapsamaktadır. Donanımsal olarak herhangi bir fiziksel bariyer motorunun elektriksel olarak tetiklenmesi (IoT röle kontrolü) mevcut proje sınırlarının dışında bırakılmış olup; proje tamamen yazılımsal analiz, veri transferi, yetkilendirme ve otonom karar destek mekanizmalarına odaklanmıştır.

Sistemin kapsamı iki ana katmanda şekillenmiştir:

- **Mobil İstemci (Android Frontend) Kapsamı:** Uygulama, cihazın kamerasından (CameraX kütüphanesi ile) sürekli olarak canlı görüntü akışı alır. Bu akış, çevrimdışı (offline) çalışan makine öğrenmesi modeli (Google ML Kit) ile analiz edilir.

  Tespit edilen metin yığınları, Türkiye Cumhuriyeti plaka standartlarına uygunluğunu denetleyen Sözdizimsel Filtre (Regex) işleminden geçirilir. Hatalı okumaları engellemek için aynı plakanın art arda üç kez tespit edilmesi şartı (kilitlenme algoritması) koşturulur.

  Plaka kilitlendiğinde, aracın anlık fotoğraf karesi kaliteden ödün verilerek sıkıştırılır ve Base64 formatında sunucuya iletilir. Arayüz, rol tabanlı yetkilendirme ile Sistem Yöneticisi ve Güvenlik Görevlisi için farklı işlevler sunar.

- **Merkezi Sunucu (Python Backend) Kapsamı:** Python ve Flask kullanılarak geliştirilen sunucu yazılımı, mobil cihazlardan gelen asenkron JSON isteklerini karşılar.

  İletilen fotoğraf verisi, veritabanını şişirmemek adına kodlanmış metinden fiziksel JPG dosyalarına dönüştürülerek sunucu dizinine kaydedilir. Plaka numarası SQLite ilişkisel veritabanında sorgulanır.

  Sistem, yetkili araca "GİRİŞ ONAYLANDI", kayıtsız araca "KAYITSIZ ARAÇ TESPİTİ", kara listedeki araca ise "YASAKLI ARAÇ ALARMI" yanıtlarını üretir. Ayrıca kullanıcı şifrelerinin güvenliği için kriptografik yöntemleri kapsar.

1.3. Literatür Taraması ve Teknolojik Arka Plan

Otomatik Plaka Tanıma Sistemleri, bilgisayarlı görü ve yapay zeka alanlarında yıllardır üzerinde çalışılan köklü bir disiplindir. Literatür incelendiğinde, geleneksel bir plaka tanıma sisteminin dört temel aşamadan oluştuğu görülmektedir: Görüntü elde etme, plaka bölgesinin tespiti, karakter bölütleme ve optik karakter tanıma.

1.3.1. Plaka Tespiti ve Görüntü İşleme Teknikleri

Geçmiş çalışmalarda plaka tespiti için Sobel veya Canny gibi kenar belirleme (edge detection) filtreleri ve morfolojik operatörler (açma, kapama, genişletme) sıklıkla kullanılmıştır. Bu geleneksel yöntemler, plakanın dikdörtgen geometrisine ve karakterler ile arka plan arasındaki kontrast farkına odaklanır. Ancak bu yöntemler; gece şartları, yağmur, kar veya far yansıması (güneş parlaması) gibi zorlu aydınlatma koşullarında yetersiz kalmaktadır.

Günümüzde ise bu geometrik yöntemlerin yerini YOLO (You Only Look Once) ve Haar-Cascade gibi derin öğrenme (Deep Learning) tabanlı erişimli sinir ağları (CNN) almıştır.

1.3.2. Optik Karakter Tanıma (OCR) Mimarisi

Plaka bölgesi izole edildikten sonra içerisindeki metinlerin makine diline çevrilmesi işlemi OCR olarak adlandırılır. Erken dönem çalışmalarda "Şablon Eşleştirme yöntemi kullanılmıştır; bu yöntemde kırpılan harf, sistem belleğindeki A'dan Z'ye harf şablonları ile piksel piksel karşılaştırılır. Ancak plakanın eğik olması veya harflerin silik olması durumunda bu yöntem tamamen çökmektedir.

Bu projede literatürdeki en modern yaklaşımlardan biri olan Uçta Yapay Zeka (Edge AI) mimarisi benimsenmiştir. Google ML Kit Vision API, bulut sunucularına veri göndermeye gerek kalmadan, doğrudan cihazın yerel işlem gücünü (NPU/CPU) kullanarak milisaniyeler içinde yüksek doğruluklu OCR işlemi gerçekleştirebilmektedir.

1.3.3. Dağıtık Mimari ve API Tabanlı Sistemler

Modern otomasyon sistemleri, monolitik (tek parça) yapılar yerine dağıtık istemci-sunucu (client-server) mimarisine doğru evrilmiştir. Literatürde, uç cihazların veri toplama görevi üstlendiği, hesaplama ve yetkilendirme kararlarının ise RESTful API'ler aracılığıyla merkezi sunuculara devredildiği sistemlerin en optimum performansı sunduğu kanıtlanmıştır.

Fırat PTS projesi de bu akademik veriler ışığında, görsel işlemlerin yükünü Android cihaza, veritabanı loglama yükünü ise Flask tabanlı Python sunucusuna dağıtarak asenkron bir iletişim köprüsü kurmuştur.

1.4. Tanımlamalar ve Kısaltmalar

Proje raporunun ilerleyen bölümlerinde kullanılacak olan teknik terimlerin, yazılım mimarisi kavramlarının ve kütüphane isimlerinin detaylı tanımları aşağıda verilmiştir:

- **PTS (Plaka Tanıma Sistemi):** Görüntü işleme teknikleri kullanılarak motorlu taşıtların tescil plakalarının otomatik olarak okunması ve dijital veriye dönüştürülmesi sistemidir.
- **OCR (Optical Character Recognition - Optik Karakter Tanıma):** Dijital bir görüntü (fotoğraf veya video karesi) üzerindeki metin karakterlerinin bilgisayar algoritmaları tarafından tanınarak makine dilindeki metin formatına (String) çevrilmesidir.
- **ML Kit (Machine Learning Kit):** Google tarafından mobil cihazlar için optimize edilmiş, makine öğrenmesi modellerinin internet bağlantısı gerektirmeksizin (on-device) cihaz üzerinde çalışmasını sağlayan akıllı kütüphanedir.
- **RESTful API (Representational State Transfer):** İstemci (Android) ile Sunucu (Python Flask) arasındaki iletişimin HTTP protokolleri (POST, GET, DELETE) üzerinden durum bilgisi tutulmadan gerçekleştirilmesini sağlayan modern web servis mimarisidir.

- ⇨ **JSON (JavaScript Object Notation):** Sistem bileşenleri arasında taşınan verilerin (Örn: plaka dizgisi, durum kodu, Base64 görsel) makineler tarafından kolayca parse edilebilmesini (ayrıştırılmasını) sağlayan hafif, esnek ve okunabilir veri değişim formatıdır.
- ⇨ **Base64 Şifreleme (Encoding):** Kameralardan yakalanan ikili (binary) görüntü verisinin, HTTP iletişim protokolü üzerinden JSON formatında bozulmadan taşınabilmesi için 64 karakterlik ASCII metin dizilerine dönüştürülmesi işlemidir.
- ⇨ **Volley:** Android işletim sisteminde ağ isteklerini asenkron (eşzamanlı olmayan) olarak arka planda yöneten, uygulamanın ana iş parçacığını dondurmasını (ANR- Application Not Responding) engelleyen Google destekli iletişim kütüphanesidir.
- ⇨ **Regex (Regular Expression - Düzenli İfadeler):** Görüntüden elde edilen ham metinlerin içinden, belirli bir matematiksel kurala (Türkiye plaka standartları olan İl Kodu - Harf Bloğu - Rakam Bloğu dizilimine) uyan parçaların algoritmik olarak süzülmesini sağlayan karakter arama şablonlarıdır.
- ⇨ **SQLite / RDBMS:** Sunucu sistemine ağır konfigürasyon yükü bindirmeyen, kurulum gerektirmeyen, verilerin yerel bir kütük dosyası (.db) içinde ilişkisel tablolar halinde saklanmasını sağlayan hafif veritabanı yönetim sistemidir.
- ⇨ **Cryptographic Hash (Kriptografik Özet Fonksiyonu):** Personel şifrelerinin veritabanında açık metin (plain text) olarak saklanması yerine, geri döndürülemez matematiksel bir diziye dönüştürülmesidir. Bu projede güvenliği artırmak için Python Werkzeug kütüphanesi kullanılmıştır.

Uç Bilişim (Edge Computing): Veri işleme faaliyetlerinin merkezi bulut sunucularda değil, verinin üretildiği yere en yakın kaynakta (bu projede Android telefonun kendi işlemcisi üzerinde) yapılması mimarisidir.

1.5. Kullanılacak Teknolojiler

Sistemin geliştirilmesinde uçtan uca (Full-Stack) mimari benimsenmiş olup aşağıdaki teknolojiler kullanılmıştır:

- **Mobil İstemci:** Android Studio IDE, Java, Jetpack CameraX.
- **Görüntü İşleme (OCR):** Google ML Kit Vision API.
- **Backend:** Python 3, Flask Web Çatısı, Werkzeug (kriptografi).
- **Veritabanı ve Ağ:** SQLite (RDBMS), Google Volley (RESTful HTTP istekleri), Ngrok, JSON ve Base64 Encoding.

2.PROJE PLANI VE YÖNETİMİ

2.1. Giriş

Yazılım mühendisliği projelerinde hedeflenen kalite standartlarına ulaşmanın ve kaynak israfını önlemenin temel şartı, kodlama evresine geçilmeden önce tüm süreçlerin titizlikle planlanmasıdır.

Fırat Üniversitesi kampüs girişleri için tasarlanan Plaka Tanıma Sistemi (PTS); uçta çalışan yapay zeka algoritmalarını (Edge AI), asenkron ağ isteklerini ve merkezi veritabanı mimarisini eşzamanlı olarak barındıran karmaşık bir otomasyon sistemidir. Bu bölümde projenin yönetimsel kısıtları, iş planı, takım yapısı, maliyet kestirim analizleri (COCOMO) ve risk yönetim stratejileri detaylandırılmıştır.

2.2. Projenin Plan Kapsamı ve Kısıtları

Geliştirilecek olan sistem, yalnızca bir mobil arayüz sunmakla kalmayıp, verilerin güvenle analiz edileceği bir backend mimarisini de kapsar.

Projenin başarıyla tamamlanabilmesi için aşağıdaki operasyonel kısıtlar göz önünde bulundurulmuştur:

- **Donanım ve İşlemci Kısıtı:** Optik Karakter Tanıma (OCR) işlemleri (Google ML Kit) sunucu yerine doğrudan mobil cihazın CPU/NPU birimleri üzerinde gerçekleştirilmektedir. Bu durum, cihazın kamera odaklama hızı ve RAM kapasitesinin (Base64 dönüşümleri için) belirli bir asgari standardı karşılamasını zorunlu kılar.
- **Ağ Bağımlılığı:** Android uç cihazı ile Python sunucusu arasındaki veri iletimi kesintisiz bir yerel ağ (WLAN) veya güvenli tünelleme servisleri (Örn: Ngrok) üzerinden sağlanmalıdır. Olası ağ kesintilerinde sistemin kilitlenmemesi için hata yönetim (Exception Handling) senaryoları planlanmıştır.

2.3. Proje Zaman-İş Planı (Gantt Şeması)

Yazılım yaşam döngüsündeki iş paketleri, projenin zamanında teslim edilebilmesi adına 8 haftalık bir geliştirme takvimine oturtulmuştur. Süreçler, ardışık ve paralel ilerleyecek biçimde aşağıdaki Gantt şeması ile kurgulanmıştır:

İş Paketleri	H1	H2	H3	H4	H5	H6	H7	H8
Gereksinim Analizi ve Literatür Taraması	Planlama							
Veritabanı (SQLite) ve API Mimarisi Tasarımı		Tasarım						
Python / Flask Sunucu (Backend) Kodlaması			Backend Geliştirme					
Android & ML Kit (OCR) Entegrasyonu				Frontend ve AI				
Sistem Doğrulama (Test) ve Optimizasyon						Test ve Hata Giderme		
Dokümantasyon (PDF) ve Canlı Kurulum							Raporlama	

Tablo 2: Proje Zaman-İş Planı (Gantt Şeması)

2.4. Proje Ekip Yapısı ve Rol Dağılımı (Full-Stack Yaklaşım)

Proje, veri tabanı tasarımından yapay zeka entegrasyonuna, web servislerinin yazımından mobil arayüz geliştirilmesine kadar tüm süreçlerin tek bir programcı tarafından yürütüldüğü "Full-Stack" bir yaklaşım ile geliştirilmiştir.

Geliştirici; gereksinim analizi evresinde "Sistem Analisti", SQLite kurgusunda "Veritabanı Yöneticisi (DBA)", arayüz ve sunucu katmanlarında ise "Yazılım Geliştirici" rollerini üstlenerek projenin iletişim maliyetlerini sıfıra indirmiştir.

2.5. Yazılım Maliyet Kestirimi (İşlev Noktası ve COCOMO Analizi)

Yazılım geliştirme maliyetlerinin ve tahmini kaynak kod satır sayılarının (LOC) bilimsel metodolojilerle belirlenmesi adına projede Temel COCOMO modeli ve İşlev Noktası analizi kullanılmıştır.

2.5.1. İşlev Noktası (AİN) Hesaplaması

Ölçüm Parametresi	Sayı	Ağırlık Faktörü	Toplam
Kullanıcı Girdi Sayısı (Login, Araç Ekle, Plaka Tarama)	4	3	12
Kullanıcı Çıktı Sayısı (Görsel Alarm, PDF Rapor, Listeler)	4	4	16
Mantıksal İç Kütükler / Tablo Sayısı (kullanicilar, araclar, gecis_kayitlari)	3	7	21
Dış Arayüz / Ekran Sayısı (Dashboard, Kayıtlar, Personel Yönetim)	5	5	25
Ana İşlev Nokta Sayısı (AİN Değeri)			**74**

Tablo 3: İşlev Noktası (AİN) Hesaplama Tablosu

2.5.2. Teknik Karmaşıklık Faktörü (TKF) ve Satır Kestirimi

Projenin gerçek zamanlı performans zorunluluğu, asenkron iletişim altyapısı ve uçta çalışan görüntü işleme gereksinimleri nedeniyle Teknik Karmaşıklık Faktörü anketi uygulanmış ve toplam TKF değeri 57 olarak hesaplanmıştır. 14 maddelik değerlendirme neticesinde elde edilen sonuçlar aşağıdaki formül ile nihai İşlev Noktası'na (İN) dönüştürülmüştür:

- **Formül:** İN = AİN x [0.65 + (0.01 x TKF)]
- **Hesaplama:** İN = 74 x [0.65 + 0.57] = 74 x 1.22 = 90.28
- **Kod Satır Sayısı:** Satır Sayısı = İN * 30
- **Sonuç:** Toplam Tahmini Satır = 90.28 * 30 ≅ 2708 LOC (Lines of Code)

Ulaşılan 2708 satırlık kod kestirimi, mobil uygulama ile arka uç API mimarisinin toplam algoritmik ağırlığını yansıtmakta olup, projenin orta-üst düzey karmaşıklık sınıfında yer aldığını kanıtlamaktadır.

2.6. Önerilen Sistemin Teknik Tanımları

PTS mimarisi iki temel bilgisayar bilimleri paradigması üzerine inşa edilmiştir:

- **Uç Bilişim (Edge Computing):** Kameralardan saniyede alınan yüksek boyutlu görüntü yığınları sunucuya gönderilmez. Optik Karakter Tanıma (OCR) doğrudan mobil cihazın kaynakları tüketilerek yapılır. Sadece ayrıştırılan anlamlı metinler

sunucuya aktarılır. Bu yaklaşım ağ bant genişliği kullanımını radikal bir biçimde düşürür.

- **RESTful Çevrim İçi İşlem (On Line Processing):** Android cihaz, tespit ettiği plakayı HTTP POST isteğiyle anlık olarak Python/Flask sunucusuna iletir ve bloklanmadan yanıtı bekler. Durumsuz (stateless) iletişim sayesinde sistem aynı anda birden fazla mobil terminalli nizamiye kapısını destekleyebilmektedir.

2.6.1. Minimum Sistem ve Donanım Gereksinimleri

Geliştirilen PTS mimarisinin sahada donma veya çökme yaşamadan stabil bir şekilde 1.5 saniyelik tepki süresiyle çalışabilmesi için öngörülen minimum gereksinimler aşağıda tablolaştırılmıştır.

Birim	Minimum Donanım İhtiyacı	Minimum Yazılım / İşletim Sistemi
Merkezi Sunucu (Backend)	2 Çekirdekli İşlemci, 4 GB RAM, Loglar ve Görseller için Min. 50 GB HDD/SSD depolama.	Windows 10 veya Ubuntu 20.04 LTS, Python 3.8 veya üzeri, SQLite 3.
Mobil İstemci (Görevli Cihazı)	Min. 3 GB RAM, Otomatik odaklanma (Auto-Focus) destekli Min. 12 MP Arka Kamera.	Android 8.0 (API Seviyesi 26 - Oreo) veya üzeri işletim sistemi.

Tablo 4: Fırat PTS Minimum Donanım ve Yazılım İhtiyaçları

2.7. Kullanılan Geliştirme Araçları (Ortamlar)

Projenin tasarlanması ve kodlanması sürecinde endüstri standartlarındaki IDE'ler ve yardımcı araçlar kullanılmıştır:

- **Android Studio (Java):** İstemci tarafındaki mobil uygulamanın (APK) derlenmesi, XML tabanlı görsel arayüzlerin tasarımı ve CameraX bileşenlerinin entegre edilmesi için kullanılan temel geliştirme ortamıdır.
- **Visual Studio Code / PyCharm:** Python dilinde Flask web servislerinin yazıldığı, veritabanı rotalarının oluşturulduğu platformdur.
- **Ngrok:** Yerel geliştirme ortamında (localhost:5000) koşan Flask sunucusunun, fiziksel Android cihaza internet üzerinden güvenli bir tünelle ulaştırılmasını sağlayan araçtır.
- **SQLite Studio:** plaka_sistemi.db isimli yerel veritabanı içindeki tabloların görsel arayüz üzerinden denetlenmesi için kullanılmıştır.

2.8. Proje Standartları ve Yaşam Döngüsü Modeli

Yapay zeka tabanlı görüntü işleme sistemleri; ışık, açı ve donanım hassasiyeti gibi çevresel faktörlere doğrudan bağımlıdır. Bu nedenle projede geleneksel doğrusal modeller yani Şelale vb. yerine, yinelemeli risk analizine olanak tanıyan **Helezonik (Spiral) Model** benimsenmiştir.

Helezonik Modelin 4 Temel Aşaması (Her Döngüde Tekrarlanır)

1. **Planlama ve Hedef Belirleme:** Bu döngüde ne yapılacağı ve hedefler belirlenir.
2. **Risk Analizi:** Olası teknik, donanımsal veya çevresel riskler masaya yatırılır ve çözüm aranır. *(Helezonik modelin en önemli ve onu diğerlerinden ayıran farkı bu aşamadır).*
3. **Geliştirme ve Test (Mühendislik):** Kodlama yapılır ve o döngünün prototipi test edilir.
4. **Değerlendirme:** Çıkan sonuç incelenir, başarılıysa bir sonraki, daha büyük bir döngüye geçilir.

Diğer Projelerden Genel Örnekler

- **Yapay Zeka Projeleri:** Örneğin bir yüz tanıma sistemi yapıyorsun. İlk döngüde sadece yüzün sınırlarını çizmeyi başarırsın (Risk: Gözlüklü insanları tanımama). İkinci döngüde gözlük riskini çözen kodlar eklersin.
- **Oyun Geliştirme:** İlk döngüde sadece karakterin yürümesini ve zıplamasını kodlarsın (Prototip). Test edersin. Riskleri görürsün (karakter duvardan geçiyor). İkinci döngüde fizik motorunu eklersin.

Senin Projende (Fırat PTS) Helezonik Model Nasıl Uygulandı?

Yapay zeka tabanlı OCR, mobil uygulama ve sunucu haberleşmesi doğası gereği risklerle dolu bir projedir. Çünkü kameranın açısı, ışık parlaması, plakanın kirliliği gibi önceden bilemeyeceğiniz çevresel riskler vardır. Bu yüzden proje tek seferde yazıp bitirilmedi; **Helezonik Model döngüleri** ile geliştirildi.

1. Birinci Döngü: Temel Görüntü İşleme (Prototip)

- **Hedef:** Kameranın açılması ve ekrandaki metinlerin okunması.
- **Risk Analizi:** Cihazın kamerasının donması veya Google ML Kit'in her gördüğü yazıyı (kampüs tabelaları) okuması riski.
- **Geliştirme:** CameraX ve ML Kit entegre edildi. Canlı akıştan metinler çekildi.
- **Değerlendirme:** Yazılımlar metni okudu ancak her şeyi plaka sanmaya başladı. Bir filtreleme sistemine ihtiyaç duyulduğu anlaşıldı ve 2. döngüye geçildi.

2. İkinci Döngü: Algoritmik Filtreleme (Regex)

- **Hedef:** Sadece Türkiye standartlarındaki araç plakalarının süzülmesi.
- **Risk Analizi:** Sıfır (0) rakamının 'O' harfiyle karışması veya kısa metinlerin sistemi yanıltması riski.
- **Geliştirme:** Metin bloklarını süzen matematiksel **Regex (Düzenli İfade)** algoritması koda eklendi. (Sadece İl Kodu - Harf - Rakam formatındakileri al).
- **Değerlendirme:** Regex başarıyla çalıştı, sadece plakalar ekrana düştü. Ancak kamera saniyede 30 kare okuduğu için aynı plakayı anında arka arkaya 30 kez buldu. Bu durumun sunucuyu çökerteceği (DDoS/Flood riski) fark edilerek 3. döngüye geçildi.

3. Üçüncü Döngü: Otonom Karar ve Kilitlenme Mekanizması

- **Hedef:** Hatalı okumaları engellemek ve sunucuyu gereksiz trafikten korumak.
- **Risk Analizi:** Ağ bant genişliğinin dolması ve yanlış bir harf okumasıyla yanlış araca alarm verilmesi.
- **Geliştirme:** "Sayaç" ve "Kilitlenme" mimarisi kuruldu. Aynı plaka art arda 3 kez aynı şekilde okunmadan (onay_siniri = 3) hedef kilitlenmedi. Kilitlenme sağlandığında fotoğraf Base64 ile sıkıştırıldı ve sistem kendisine 3 saniyelik bir bekleme (uyku) süresi koydu.
- **Değerlendirme:** İstemci tarafı mükemmel hale geldi, sistem kararlı çalıştı. Artık veritabanı (Sunucu) entegrasyonu için 4. döngüye geçildi.

4. Dördüncü Döngü: Sunucu Haberleşmesi ve Acil Durum Uyarıları

- **Hedef:** Okunan plakanın Python (Flask) sunucusuna gönderilip yetki kontrolünün yapılması ve görevliye bildirilmesi.
- **Risk Analizi:** İnternet bağlantısı koptuğunda mobil uygulamanın çökmesi (ANR - Application Not Responding) riski.
- **Geliştirme:** Volley asenkron kütüphanesi kullanılarak API iletişimi sağlandı. Ağ kopmalarına karşı TimeoutError (Zaman aşımı) yakalama blokları (try-catch) eklendi. Sunucudan dönen sonuca göre arayüzün rengi (Kırmızı/Yeşil) değiştirildi ve Titreşim/Ses donanımları tetiklendi.

Sonuç olarak; Helezonik model kullanılması sayesinde, ilk döngüde yaşanabilecek "sunucu çökmesi" veya "yanlış plaka okuma" gibi devasa riskler, adım adım test edilerek ve her döngüde kodlar rafine edilerek (iyileştirilerek) kusursuz bir otomasyon sistemine dönüştürülmüştür.

Geliştirme süreci boyunca; ML Kit kütüphanesi ile plakalar okutulmuş, sıfır rakamının "O" harfiyle karışması gibi tespit edilen riskler değerlendirmeye alınarak koda *Düzenli İfade*

(Regex) filtreleri eklenmiştir. Testlerin sonucuna göre kod sürekli rafine edilerek bir sonraki döngüye geçilmiştir.

2.9. Kalite, Konfigürasyon ve Risk Yönetim Planları

Risk No	*Risk Tanımı*	*Olasılık*	*Etki*	*Alınan Önlem / Çözüm Stratejisi*
R1	Kameranın gece/karanlıkta plakayı okuyamaması	Orta	Yüksek	Arayüze "Manuel Araç Ekleme" butonu konularak sistemin kilitlenmesi önlendi.
R2	Kampüs internetinin kopması (Sunucuya ulaşılamaması)	Düşük	Kritik	Volley kütüphanesine "TimeoutError" eklendi. Uygulamanın çökmesi (ANR) engellendi.
R3	Veritabanının fotoğraflarla şişip sunucuyu çökertmesi	Yüksek	Yüksek	Fotoğraflar veritabanı yerine sunucu diskinde .jpg olarak tutuldu.
R4	Aynı plakanın saniyede 30 kez sunucuya gönderilmesi	Yüksek	Orta	Sistemde "3 kez okuma ve kilitlenme (Sayaç)" algoritması geliştirildi.

Tablo 5: Kalite, Konfigürasyon ve Risk Yönetim Planı Tablosu

Sistemin 7/24 kesintisiz çalışması gereken kampüs nizamiyelerinde çökmesini engellemek için proaktif risk yönetimi uygulanmıştır. Olası ağ gecikmeleri veya sunucu kopmaları durumunda Android cihazın "Uygulama Yanıt Vermiyor (ANR)" hatası üretmemesi için iletişim katmanına (Volley kütüphanesi) *RetryPolicy (Yeniden Deneme Politikası)* kuralları tanımlanmış, zaman aşımı süreleri 10 saniye (10.000 ms) olarak sınırlandırılmıştır.

Kod kalitesini artırmak için nesne yönelimli programlamanın "Kapsülleme" ilkelerine uyulmuş, modüller birbirine gevşek bağlı şekilde tasarlanarak sistemin genişletilebilirliği güvence altına alınmıştır.

2.10. Veri Akış Diyagramı (DFD - 0. Seviye Bağlam Diyagramı)

Fırat PTS projesinin dış varlıklar ile olan veri alışverişini soyut bir düzlemde göstermek amacıyla 0. Seviye (Context) Veri Akış Diyagramı oluşturulmuştur. Bu diyagramda sistem tek bir ana süreç (0) olarak ele alınmış, Güvenlik Görevlisi ve Sistem Yöneticisi aktörlerinin sisteme hangi veri paketlerini gönderdiği ve sistemden hangi çıktıları aldığı modellenmiştir.



Şekil 1: Veri Akış Diyagramı (DFD - 0. Seviye Bağlam Diyagramı)

3.GEREKSİNİM ANALİZİ

3.1. Mevcut Sistem İncelemesi

Sistem çözümleme evresi, yazılımı geliştirilecek olan otomasyonun gereksinimlerinin analiz edilmesi, mevcut yapının darboğazlarının tespit edilmesi ve bu sorunlara çözüm sunacak mantıksal modellerin ortaya konulması sürecidir. Fırat Üniversitesi nizamiyelerindeki mevcut araç kontrol mekanizmaları, projenin çıkış noktasını oluşturduğu için detaylı bir analize tabi tutulmuştur.

3.1.1. Mevcut Örgüt Yapısı ve İşlevsel Darboğazlar

Kampüs nizamiyelerindeki güvenlik süreci; üniversite idaresi (Rektörlük), güvenlik amirliği ve sahada görev yapan güvenlik görevlilerinden oluşan hiyerarşik bir örgüt yapısı ile yönetilmektedir. Mevcut işlevsel modelde, kural olarak her aracın nizamiyede durdurulması, sürücünün kurum kimliğini veya taşıt pulunu ibraz etmesi gerekmektedir. Özellikle sabah 08:00 ve akşam 17:00 sularında tek bir güvenlik görevlisinin yüzlerce aracı manuel olarak kontrol etmeye çalışması, insan doğası gereği fiziksel yorgunluğa, dikkat dağınıklığına ve kampüs dışına kadar uzanan ciddi trafik kuyruklarına (işlevsel darboğaz) neden olmaktadır.

3.1.2. Mevcut Veri Modeli

Mevcut sistemdeki veri modeli yapılandırılmamış ve dağınık bir formdadır. Kampüse dışarıdan giren misafir araçların veya şüpheli durumların log kayıtları, güvenlik görevlileri tarafından fiziksel nöbet defterlerine veya lokal bilgisayarlardaki bağımsız Excel dosyalarına manuel olarak işlenmektedir. Bu durum, veri bütünlüğünü bozmakta ve merkezi bir bilgi sisteminin kurulmasını imkansız kılmaktadır.

3.1.3. Var Olan Sistemin Değerlendirilmesi ve Eksiklikleri

Mevcut manuel sistem; izlenebilirlik, sorgulanabilirlik ve veri güvenliği açılarından yetersizdir. Geriye dönük adli veya idari bir inceleme gerektiğinde, binlerce sayfalık fiziksel defterlerin taranması günler sürmektedir.

Hatalı plaka kaydı yazılması riski yüksektir. Ayrıca fiziksel kayıtların yetkisiz kişilerin erişimine açık olması, Kişisel Verilerin Korunması Kanunu (KVKK) ihlali risklerini barındırmaktadır.

3.2. Gereksinilen Sistemin Mantıksal Modeli

Mevcut sistemin zafiyetlerini gidermek üzere tasarlanan Fırat PTS yazılımı, insan inisiyatifini en aza indiren otonom bir bilgisayarlı görü merkezi veri yönetimi modeli sunar.

3.2.1. Sistem Aktörleri ve Roller

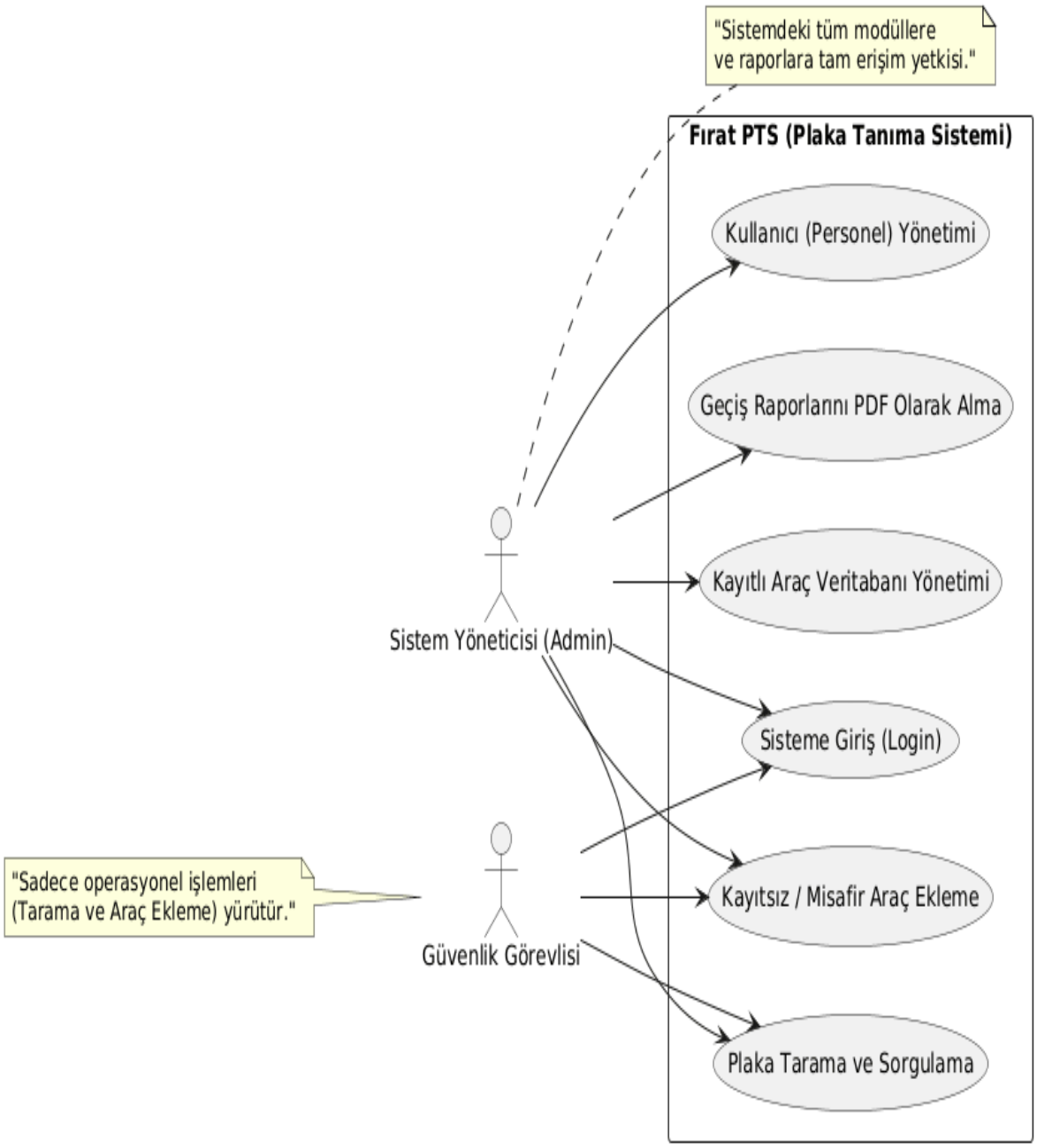
Sistemin güvenliğini sağlamak için Rol Tabanlı Erişim Kontrolü (RBAC - Role Based Access Control) kullanılmıştır. Sistemin mantıksal modelinde iki ana aktör bulunur:

- **Sistem Yöneticisi (Admin):** Yazılımın tüm modüllerine erişim yetkisi olan süper kullanıcıdır. Sisteme yeni güvenlik görevlisi ekleyebilir, silebilir, kayıtlı araçların veri tabanını (Akademik, İdari, Öğrenci) yönetebilir ve geçmiş logları PDF olarak dışa aktarabilir.
- **Kampüs Güvenlik Görevlisi (Gorevli):** Yalnızca nizamiyede aktif operasyonu yöneten aktördür. Mobil uygulama üzerinden sisteme giriş yaparak plaka taraması gerçekleştirir. Gelen "Onaylı" veya "Yasaklı" bildirimlerine göre aksiyon alır. Sisteme misafir/yabancı araç ekleyebilir, ancak sistemden personel silme veya ekleme gibi idari modüllere (Yetki kısıtlaması nedeniyle) erişemez.

3.2.2. Kullanım Senaryoları (Use-Case)

Yazılımın temel Use-Case senaryoları mantıksal olarak şu şekilde özetlenebilir:

- **UC-1 (Sisteme Giriş):** Aktör, kullanıcı adı ve şifresiyle sisteme yetkili giriş (Login) talebi gönderir. API, kimlik doğrulamasını (Hash kontrolü) yapar.
- **UC-2 (Plaka Tarama):** Görevli, mobil cihazın kamerasını araca yöneltir. Sistem arka planda yapay zeka ile metni okur ve veritabanına sorgu atar.
- **UC-3 (Yasaklı Araç Bildirimi):** Sistem taranan aracın kara listede olduğunu tespit ederse, donanımsal alarmları (titreşim ve ses) tetikleyerek görevliye "Güvenlik İhlali" mesajı sunar.
- **UC-4 (Geçici Araç Kaydı):** Tespit edilen araç sistemde yoksa (Yabancı), sistem otomatik olarak görevliyi "Araç Ekleme" ekranına yönlendirir.

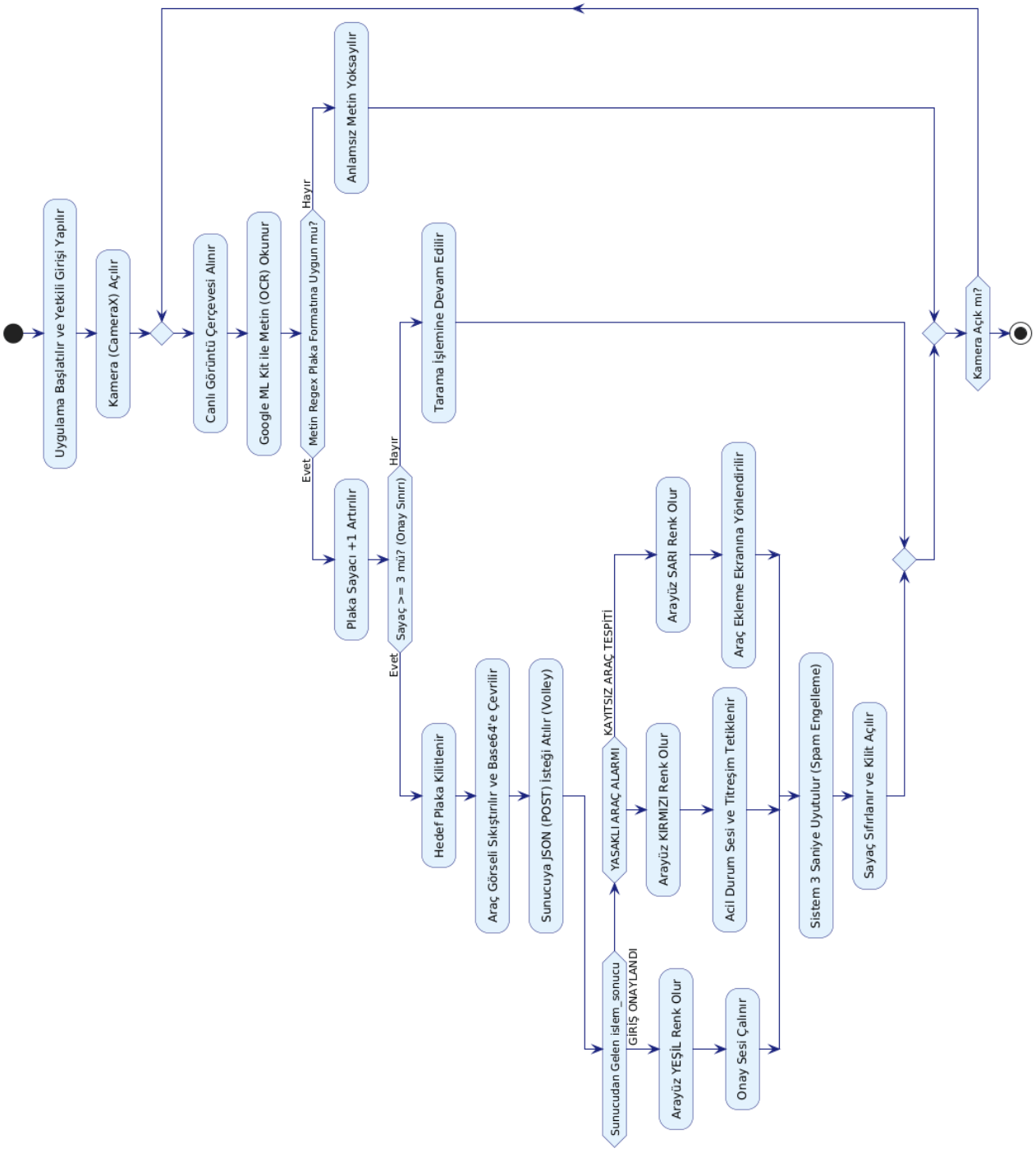


Şekil 2: Kullanım Senaryoları (Use-Case) Diyagramı

3.2.3. Genel Sistem Akışı

Sistemin anlık bir operasyon (geçiş) sırasındaki işlevsel algoritması adım adım şu şekildedir:

1. **Görüntü Yakalama:** CameraX, canlı akıştan ardışık kareler çeker.
2. **Yapay Zeka (OCR) Analizi:** Görüntü karesi, cihaz üzerindeki Google ML Kit motoruna beslenir ve ekrandaki metinler (plaka, tabela vb.) çıkarılır.
3. **Sözdizimsel Filtreleme (Regex):** Bulunan metinler, Türkiye plaka standardı şablonuna (Örn: `(([0-9]{2})\s*)?([A-Z]{1,3})\s*([0-9]{2,4})`) tabi tutularak anlamsız kelimeler elenir.
4. **Doğrulama ve Kilitlenme:** Hatalı okumayı engellemek için, aynı plakanın ardışık olarak en az 3 kez tespit edilmesi (onay_siniri = 3) beklenir. Sayaca ulaşılırsa hedef kilitlenir.
5. **Sıkıştırma ve İletişim:** Kilitlenme anındaki araç fotoğrafı JPEG formatında sıkıştırılıp Base64 metnine dönüştürülür. Plaka metni ile beraber JSON paketi halinde sunucuya POST edilir.
6. **Sunucu İşlemi ve Loglama:** Python Backend, SQLite'ta arama yapar, aracın durumuna göre işlem sonucunu (GİRİŞ ONAYLANDI veya YASAKLI ARAÇ ALARMI) belirler. Görseli disk üzerine kaydeder ve veritabanına log işler.
7. **Arayüz Bildirimi:** İstemci, sunucudan dönen JSON paketindeki `durum_kodu` değişkenini işleyerek arayüzü günceller (Yeşil/Kırmızı ekran) ve sistemi 3 saniye sonra yeni araç için sıfırlar.



Şekil 3: Genel Sistem Akış Diyagramı

3.2.4. Bilgi Sistemleri ve Nesneler Arası İletişim

Sistemde veri transferi Asenkron İletişim prensibiyle gerçekleşir. Android üzerindeki *DashBoardActivity* nesnesi, arayüzü dondurmadan arka planda bir iletişim iş parçacığı başlatarak sunucu ile haberleşir.

Bilgi nesneleri (Kullanıcı, Araç ve Log) JSON (JavaScript Object Notation) formatında sarmalanarak platform bağımsız ve düşük boyutlu bir taşıma protokolüne oturtulmuştur.

3.2.5. Başarım (Performans) Gerekleri

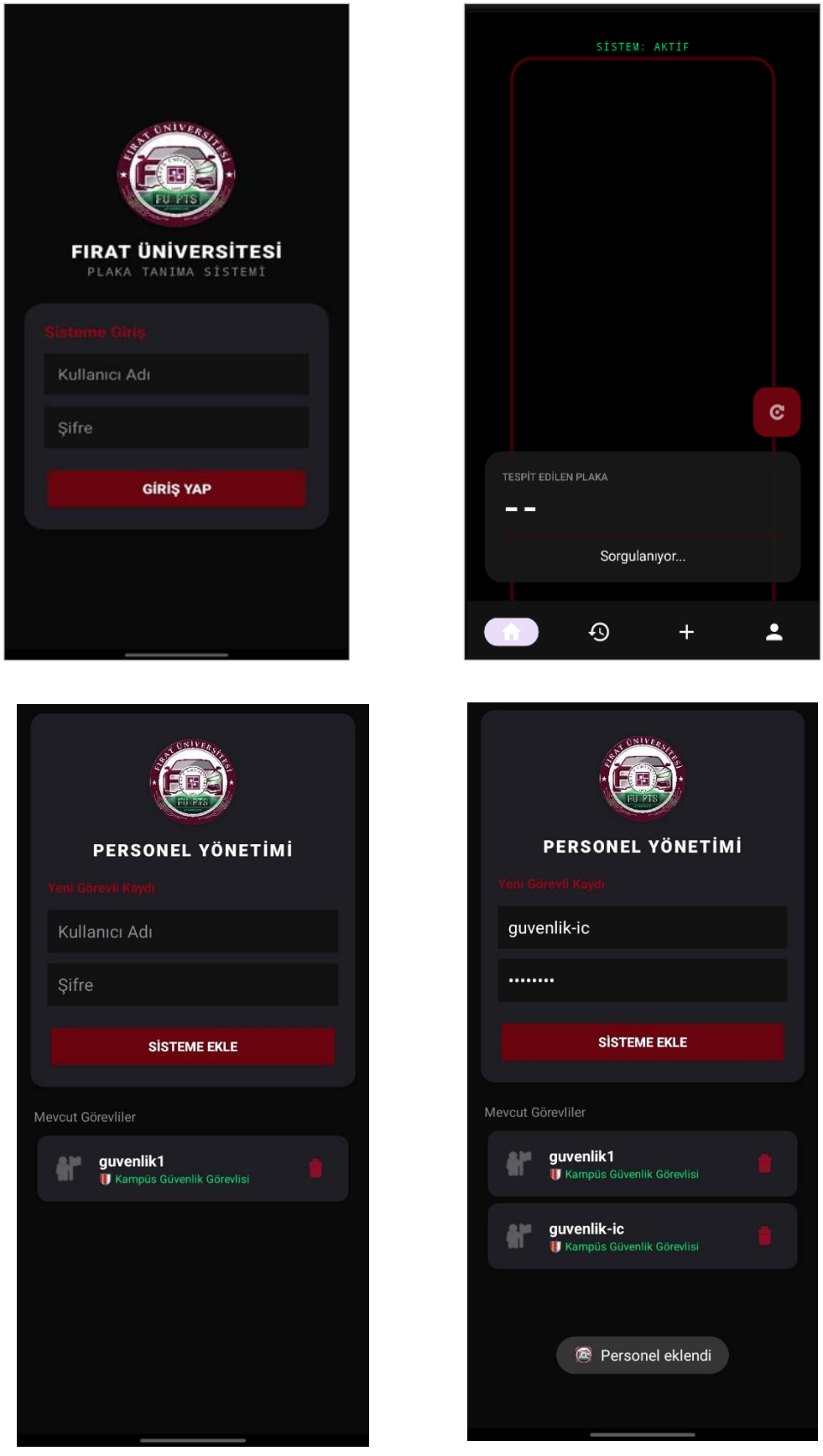
Trafik akışını yavaşlatmamak adına sistemin Maksimum 1.5 Saniye tepki süresine (Response Time) sahip olması zorunludur. Bu başarıma ulaşmak için;

- Görüntüler ağa gönderilmeden önce mobil cihazda %70 kaliteye (Quality: 70) sıkıştırılarak ağ gecikmesi önlenmiştir.
- Veritabanı ilişkileri optimize edilerek Python üzerindeki SQL sorgularının milisaniyeler düzeyinde sonuçlanması sağlanmıştır.

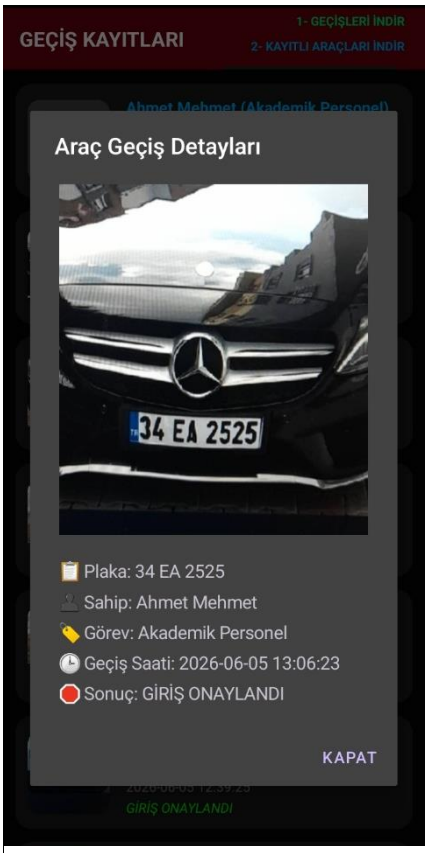
3.3. Arayüz (Modül) Gerekleri

3.3.1. Mobil Kullanıcı Arayüzü (UI/UX) Gereksinimleri

Uygulama arayüzü, nizamiyede gece vardiyasında da görev yapan personelin göz sağlığını korumak ve pil tüketimini düşürmek amacıyla Karanlık Tema (Dark Mode) standartlarında tasarlanmıştır. Ergonomi gereği canlı kamera akışı ekranın üst yarısına, operasyonel sonuçlar ise alt yarısına çok büyük ve okunabilir puntolarla yerleştirilmiştir. Navigasyon kolaylığı için Alt Menü (BottomNavigationView) kullanılmış, sisteme giren aktörün rolüne ("gorevli" veya "admin") göre bu menüdeki yetkisiz sekmeler gizlenmiştir.



Şekil 4: Mobil Kullanıcı Arayüzü Ekran Görüntüleri



Şekil 5: Donanım Arayüzü ve Alarm Ekranı

3.3.2. Donanım Arayüzü (Kamera, Ses ve Titreşim Motorları)

Sistem, mobil cihazın temel donanımları ile doğrudan entegre çalışır. Kamera donanımı üzerinden yüksek yenileme hızında (FPS) görüntü alınır. Operatörün ekranı sürekli izleme zorunluluğunu ortadan kaldırmak için cihazın donanımsal Titreşim Motoru (Vibrator) ve Ses Motoru (ToneGenerator) arayüzlere entegre edilmiştir.

Yasaklı bir araç tespit edildiğinde acil durum frekansı ve yüksek yoğunluklu titreşim dalgaları oluşturularak personelin dikkati derhal konuya çekilir.

3.3.3. API ve İletişim Arayüzü

Sistem birimleri arasındaki haberleşme RESTful web servisleri (API) kullanılarak gerçekleştirilmiştir. Sunucu uygulamasında `/api/plaka-sorgula`, `/api/giris`, `/api/araclar` gibi modüler iletişim uç noktaları tanımlanmıştır. API istekleri, Google Volley kütüphanesi tarafından yönetilmektedir. Ayrıca yerel geliştirme süreçlerinde mobil ağ üzerinden yerel sunucuya ulaşılabilmesi için *Ngrok* tünelleme servisinden yararlanılmış, bu amaçla API istek başlıklarına özel tünel parametreleri (ngrok-skip-browser-warning) eklenerek kesintisiz bir iletişim köprüsü kurulmuştur.

4.SİSTEM TASARIMI VE MİMARİSİ

4.1. Genel Tasarım ve Sistem Mimarisi

Sistem tasarımı aşaması, çözümleme evresinde oluşturulan mantıksal gereksinimlerin ve kavramsal modellerin, kodlanabilir fiziksel mimarilere dönüştürülme sürecidir. Fırat Üniversitesi nizamiyeleri için tasarlanan Plaka Tanıma Sistemi (PTS), modern yazılım mühendisliği paradigmalarına uygun olarak çok katmanlı bir yapıda kurgulanmıştır.

4.1.1. Uç Bilişim (Edge Computing) Yaklaşımı

Projenin en yenilikçi mimari kararlarından biri Uç Bilişim (Edge Computing) yaklaşımının benimsenmesidir. Geleneksel güvenlik kamera sistemlerinde, kameradan alınan yüksek çözünürlüklü video akışı analiz edilmek üzere sürekli olarak merkezi sunucuya gönderilir. Bu durum yerel ağda devasa bir bant genişliği tüketimine ve sunucu tarafında ağır bir işlemci (CPU/GPU) darboğazına neden olur.

Fırat PTS projesinde bu sorunu aşmak için "görüntü işleme" görevi sunucudan alınarak verinin üretildiği uca, yani güvenlik görevlisinin mobil cihazına verilmiştir. Google ML Kit Vision API, buluta veri göndermeden tamamen cihazın yerel donanım ivmelendiricilerini kullanarak saniyeler içinde Optik Karakter Tanıma (OCR) işlemini gerçekleştirir. Sunucuya video akışı değil, sadece filtrelenmiş anlamlı plaka metni ve kilitlenme anındaki tek bir sıkıştırılmış kare iletilir.

4.1.2. İstemci - Sunucu (Client-Server) Mimarisi

Sistem, görev dağılımının net bir şekilde yapıldığı dağıtık bir İstemci-Sunucu mimarisine sahiptir. İstemci konumundaki Android cihaz; veriyi toplamak, yapay zeka ile analiz etmek ve kullanıcıya görsel/işitsel dönüt (Kırmızı/Yeşil ekran) vermekle yükümlüdür.

Sunucu konumundaki Python (Flask) mimarisi ise; gelen HTTP isteklerini karşılamak, veritabanında yetki sorgulaması yapmak, log kayıtlarını tutmak ve asenkron JSON formatında yanıt üretmekle görevlidir. Bu durumsuz (Stateless) RESTful mimari, ilerleyen aşamalarda sisteme yeni nizamiyelerin (yeni istemcilerin) sunucuyu yormadan kolayca eklenmesine (Ölçeklenebilirlik - Scalability) olanak tanır.

4.2. Veritabanı ve Veri Modeli Tasarımı

4.2.1. SQLite Veritabanı Seçim Gerekçeleri

Fırat PTS projesinde ilişkisel veritabanı yönetim sistemi (RDBMS) olarak SQLite tercih edilmiştir. Bu seçimin temel mimari gerekçesi, SQLite'ın "Sunucusuz (Serverless)" ve "Sıfır Yapılandırma (Zero-Configuration)" özelliklerine sahip olmasıdır.

Oracle, MySQL veya PostgreSQL gibi ağır veritabanları, arka planda sürekli çalışan ve yüksek RAM tüketen bağımsız servislere ihtiyaç duyarken; SQLite tüm veritabanı kütüğünü işletim sistemi üzerinde tek bir yerel dosya (plaka_sistemi.db) olarak tutar. Bu sayede sistem, düşük donanımlı standart bilgisayarlarda veya tek kartlı mikrodenetleyicilerde dahi yüksek performansla çalışabilmektedir.

4.2.2. Tablo Tanımları

Veri modeli, Veritabanı Normalizasyon kurallarına uygun olarak tasarlanmış ve üç temel tabloya bölünmüştür:

- **KULLANICILAR Tablosu:** Sistemi yönetecek olan personelin verilerini tutar. id (Primary Key, AutoIncrement), username, password_hash (kriptografik güvenlik için) ve yetki ayrımını sağlayan role sütunlarından oluşur.

Rows: 2 — Filter 2 rows...

	id	kullanici_...	sifre_hash	rol
1	1	yonetici	scrypt:32768:8:1$ilelQlDG7nVUuudh$d30b513e7385323...	admin
2	2	guvenlik1	scrypt:32768:8:1$nD7dlNw0dbfG5sdz$2a21568ae45143...	gorevli
3				

Şekil 6: KULLANICILAR Tablosunun Veri Yapısı ve Sütunları

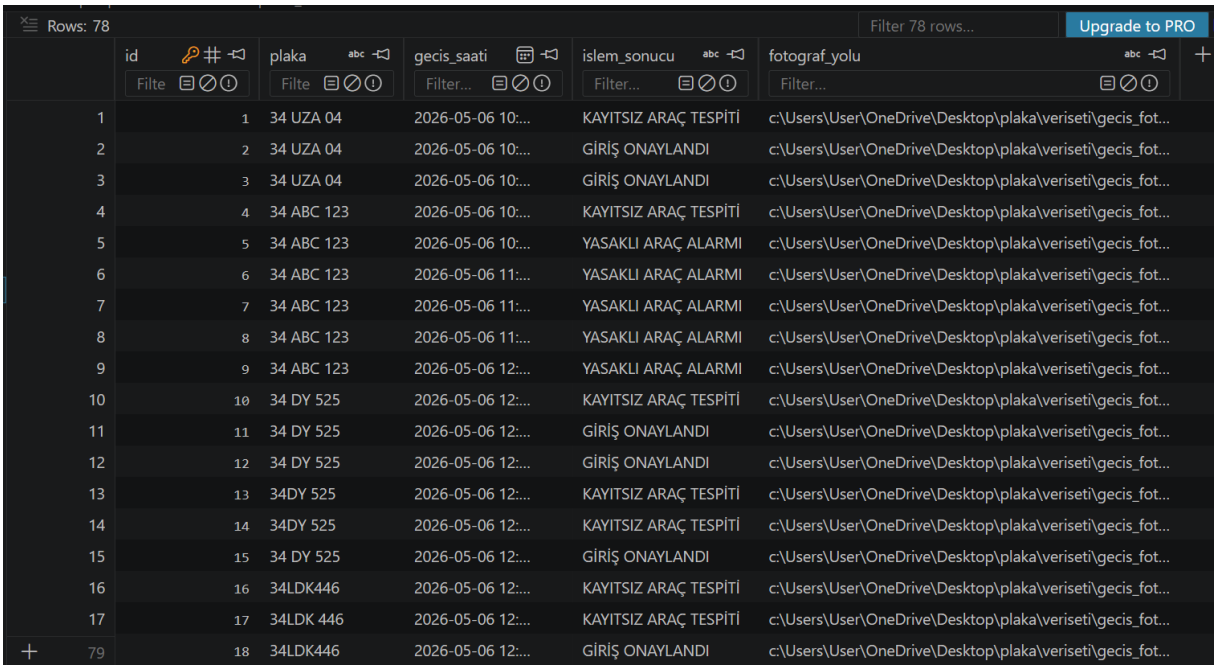
- **ARACLAR Tablosu:** Kampüse giriş yetkisi tanımlanan araçların tutulduğu ana tescil tablosudur. id (PK), plaka, sahip_ad ve aracın sisteme giriş iznini belirleyen statü (ONAYLI / YASAKLI) alanlarını barındırır.

Rows: 15 — Filter 15 rows...

	id	plaka	sahip	kategori	gecis_izni	kayit_tarihi
1	1	34 UZA 04	ali veli	Misafir	ONAYLI	2026-05-06 10:...
2	2	34 ABC 123	ahmet aaa	Misafir	YASAKLI	2026-05-06 10:...
3	3	34 DY 525	ayse fatma	Öğrenci	ONAYLI	2026-05-06 12:...
4	4	34LDK446	mahir mete	Akademik Personel	ONAYLI	2026-05-06 12:...
5	5	34 HGJ 170	aa bb	Akademik Personel	ONAYLI	2026-05-06 13:...
6	6	54 ZS 063	Selman Yılmaz	Akademik Personel	ONAYLI	2026-05-16 09:...
7	7	34 MJJ 572	Melisa Doğan	İdari Personel	ONAYLI	2026-05-16 12:...
8	8	34KOK379	Nevzat Mete	Misafir	ONAYLI	2026-05-16 14:...
9	9	34ABC123	yasak	Akademik Personel	YASAKLI	2026-05-16 14:...
10	10	34 EA 2525	Ahmet Mehmet	Akademik Personel	ONAYLI	2026-05-16 14:...
11	11	34 PK 43	Pelin Kaya	Akademik Personel	YASAKLI	2026-05-16 14:...
12	12	34 PY 2115	Yağmur Kaya	Öğrenci	ONAYLI	2026-05-16 14:...
13	13	34 LDK 446	mahir mete	Öğrenci	ONAYLI	2026-05-16 19:...
14	14	34 KOK 379	kemal kurtuluş	Akademik Personel	ONAYLI	2026-05-18 10:...
15	15	POA 34	sjdjsmnd	Akademik Personel	YASAKLI	2026-05-18 10:...

Şekil 7: ARACLAR Tablosunun Fiziksel Görünümü

- **GECIS_KAYITLARI Tablosu:** Nizamiyedeki anlık trafik akışının loglandığı tablodur. log_id (PK), plaka, tarih_saat (Zaman damgası), sonuc (İşlem durumu) ve olaya ait kanıt niteliğindeki görselin diskteki yolunu tutan fotograf_path sütunlarını içerir.



Rows: 78 — Filter 78 rows... — Upgrade to PRO

	id	plaka	gecis_saati	islem_sonucu	fotograf_yolu
1	1	34 UZA 04	2026-05-06 10:...	KAYITSIZ ARAÇ TESPİTİ	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
2	2	34 UZA 04	2026-05-06 10:...	GİRİŞ ONAYLANDI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
3	3	34 UZA 04	2026-05-06 10:...	GİRİŞ ONAYLANDI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
4	4	34 ABC 123	2026-05-06 10:...	KAYITSIZ ARAÇ TESPİTİ	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
5	5	34 ABC 123	2026-05-06 10:...	YASAKLI ARAÇ ALARMI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
6	6	34 ABC 123	2026-05-06 11:...	YASAKLI ARAÇ ALARMI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
7	7	34 ABC 123	2026-05-06 11:...	YASAKLI ARAÇ ALARMI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
8	8	34 ABC 123	2026-05-06 11:...	YASAKLI ARAÇ ALARMI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
9	9	34 ABC 123	2026-05-06 12:...	YASAKLI ARAÇ ALARMI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
10	10	34 DY 525	2026-05-06 12:...	KAYITSIZ ARAÇ TESPİTİ	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
11	11	34 DY 525	2026-05-06 12:...	GİRİŞ ONAYLANDI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
12	12	34 DY 525	2026-05-06 12:...	GİRİŞ ONAYLANDI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
13	13	34DY 525	2026-05-06 12:...	KAYITSIZ ARAÇ TESPİTİ	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
14	14	34DY 525	2026-05-06 12:...	KAYITSIZ ARAÇ TESPİTİ	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
15	15	34 DY 525	2026-05-06 12:...	GİRİŞ ONAYLANDI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
16	16	34LDK446	2026-05-06 12:...	KAYITSIZ ARAÇ TESPİTİ	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
17	17	34LDK 446	2026-05-06 12:...	KAYITSIZ ARAÇ TESPİTİ	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...
79	18	34LDK446	2026-05-06 12:...	GİRİŞ ONAYLANDI	c:\Users\User\OneDrive\Desktop\plaka\veriseti\gecis_fot...

Şekil 8: Geçiş Kayıtları Tablosunun Fiziksel Görünümü

4.2.3. Varlık-İlişki (E-R) Diyagramı ve Şemalar

Veritabanı tabloları arasındaki mantıksal ve fiziksel bağıntılar Varlık-İlişki diyagramı ile modellenmiştir. Kullanıcıların (Admin/Görevli) araçlar üzerinde yönetici yetkisi, araçların ise geçiş kayıtları üzerinde bire-çok (1:N) ilişkisi bulunmaktadır.

4.2.4. Veri Sözlüğü (Data Dictionary)

Veritabanında kullanılan temel özniteliklerin (attributes) tip ve boyut tanımlamaları aşağıdaki veri sözlüğünde standartlaştırılmıştır:

- **id:** Tamsayı (INTEGER), Otomatik artan (AUTOINCREMENT), Birincil Anahtar (PK).
- **plaka:** Metin (VARCHAR), Maksimum 20 karakter, Boş geçilemez (NOT NULL).
- **islem_sonucu:** Metin (VARCHAR), Maksimum 50 karakter (Örn: "GİRİŞ ONAYLANDI").
- **gecis_saati:** Tarih/Zaman (DATETIME), Varsayılan değer (CURRENT_TIMESTAMP).
- **fotograf_yolu:** Metin (VARCHAR), Maksimum 255 karakter, Sunucu disk yolu.

4.3. Dosya Sistemi ve Veri Depolama Stratejisi

4.3.1. Görüntülerin Veritabanı Yerine Disk Üzerinde Saklanması

Yazılım mimarisinde büyük ikili verilerin (Resim, Video vb.) veritabanı içinde BLOB (Binary Large Object) veri tipiyle tutulması, zamanla veritabanı kütüğünün devasa boyutlara ulaşmasına ve arama (Select) sorgularının yavaşlamasına neden olur.

Bu mimari riski önlemek için Fırat PTS projesinde fotoğraflar veritabanında tutulmaz. Görüntüler işletim sisteminin dosya dizininde (Hard Disk / SSD) fiziksel bir dosya (.jpg) olarak saklanır, veritabanındaki fotograf_path hücresine ise yalnızca bu dosyanın bulunduğu dizin yolu (Metin/String olarak) yazılır. Bu strateji veri tabanının milisaniyeler seviyesinde hızlı kalmasını sağlar.

4.3.2. Base64 Encoding ile Görüntü Taşıma İşlemleri

Mobil cihaz kamerasından çekilen fiziksel bir fotoğrafın (Bitmap) ağ üzerinden metin tabanlı JSON paketleriyle gönderilebilmesi için şifrelenmesi gerekmektedir. Projede bu işlem "Base64 Encoding" algoritması ile gerçekleştirilir.

İstemci tarafında fotoğraf sıkıştırılıp 64 karakterlik ASCII metin dizisine çevrilir. Sunucu tarafında (Python) ise bu uzun metin dizisi tekrar çözülerek ikili formata dönüştürülür ve disk üzerine resim dosyası olarak kaydedilir.

4.3.3. Sunucu Dizin (Directory) Yapısı ve İsimlendirme Standartları

PTS sunucusu hiyerarşik bir dizin yapısına sahiptir. Verilerin üzerine yazılmasını (Overwrite) ve isim çakışmalarını (Collision) engellemek için, fotoğraflar ana dizin altındaki gecis_fotograflari klasörüne zaman damgasıyla kaydedilir.

İsimlendirme standardı Plaka_YılAyGün_SaatDakikaSaniye.jpg (Örn: 23FU123_20260602_143000.jpg) olarak belirlenmiştir. Bu mutlak yol (Absolute Path) yapısı, adli bilişim incelemelerinde kolaylık sağlar.

4.3.4. İşletim Sistemi Dosya Yapıları ve PTS Sunucusu Uyumu

Sunucu hard diskinde verilerin güvenli okunup yazılabilmesi için uygun blok (cluster) boyutlarına sahip modern dosya sistemlerine ihtiyaç vardır. Eski nesil FAT16 veya FAT32 dosya sistemleri tek bir klasör içindeki dosya sayısında limitasyonlara ve maksimum 4GB dosya boyutu sınırına sahip oldukları için sistemin çökmesine neden olabilirler. Bu nedenle PTS sunucusunun, Linux tabanlı ise Inode mimarisini kullanan EXT4, Windows tabanlı ise NTFS dosya sistemi üzerinde konumlandırılması kararlaştırılmıştır.

4.3.5. Kaynak Kod Dizin (Klasör) Hiyerarşisi

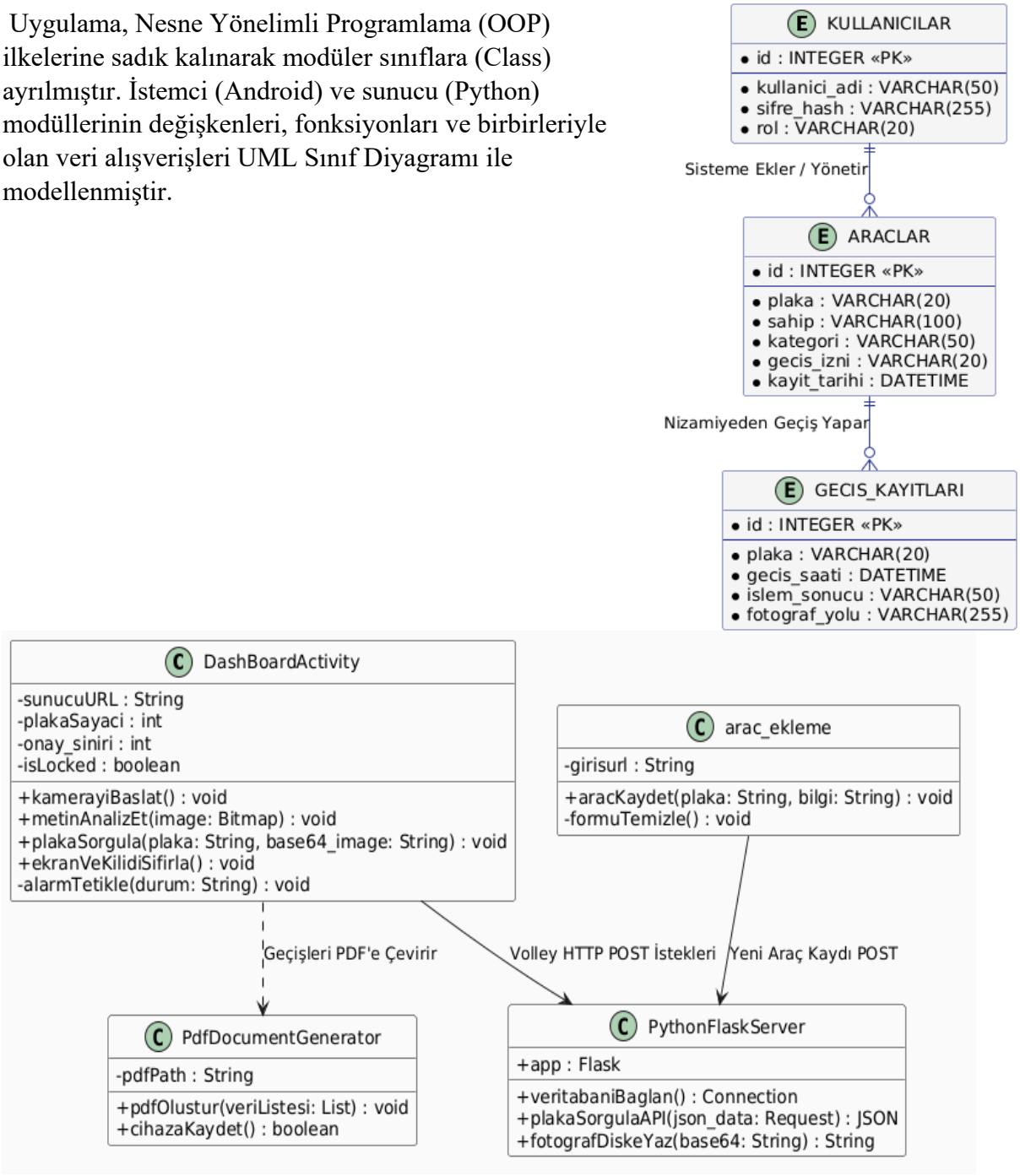
Projenin teslim edilen kaynak kodlarının çalışma dizini yapısı ve dosyaların görevleri aşağıdaki ağaç yapısında (Tree Structure) standartlaştırılmıştır:

```
FIRAT_PTS/
│
├── Backend_Python_Sunucu/
│   ├── app.py
│   ├── plaka_sistemi.db
│   ├── requirements.txt
│   └── gecis_fotograflari/
│       ├── 23FU123_2026...jpg
│       └── 34ABC12_2026...jpg
│
└── Frontend_Android_Uygulama/
    ├── app/src/main/java/.../
    │   ├── DashBoardActivity.java
    │   ├── LoginActivity.java
    │   ├── AracEkleActivity.java
    │   └── PdfDocumentGen.java
    └── app/src/main/res/
        ├── layout/
        └── raw/
```

4.4. Süreç Tasarımı ve Uygulama Modülleri

4.4.1. Modüller Arası İlişkiler (UML Sınıf / Class Diyagramı)

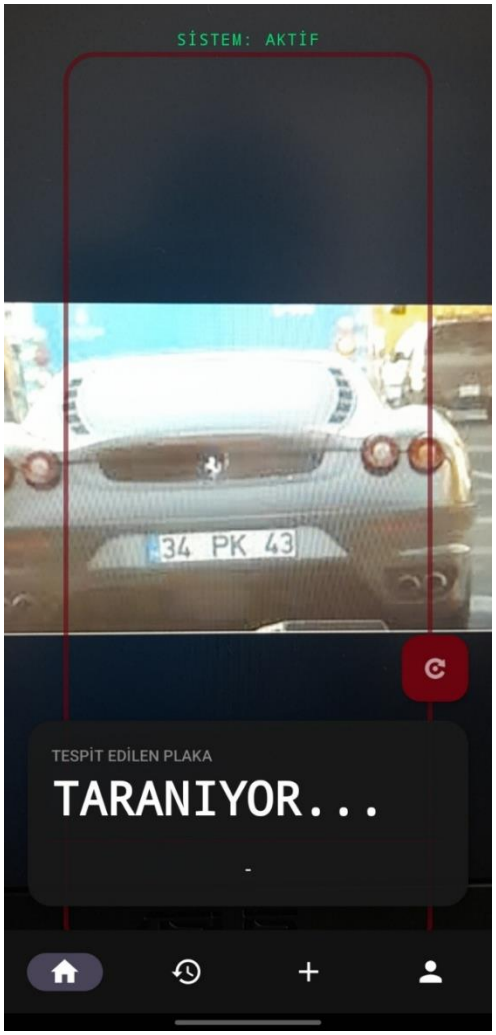
Uygulama, Nesne Yönelimli Programlama (OOP) ilkelerine sadık kalınarak modüler sınıflara (Class) ayrılmıştır. İstemci (Android) ve sunucu (Python) modüllerinin değişkenleri, fonksiyonları ve birbirleriyle olan veri alışverişleri UML Sınıf Diyagramı ile modellenmiştir.



Şekil 9: Modüller Arası İlişkiler UML Sınıf Diyagramı

4.4.2. Plaka Okuma ve Doğrulama Modülü

Sistemin ana işletim merkezidir. Bu modül; CameraX ile donanıma bağlanır, ML Kit ile OCR işlemlerini yürütür ve elde edilen metinleri Türkiye plaka formatına göre Regex (Düzenli İfadeler) algoritmasıyla süzer. Hatalı okumaları engellemek için tasarlanan "3 Kez Kilitlenme" döngüsü bu modülde işletilir. Kilitlenme sağlandığında Volley kütüphanesini tetikleyerek asenkron HTTP POST isteğini başlatır ve dönen JSON sonucuna göre arayüzdeki renkleri (Kırmızı/Yeşil/Sarı) ve donanım motorlarını (Titreşim, Ses) otonom olarak yönetir.

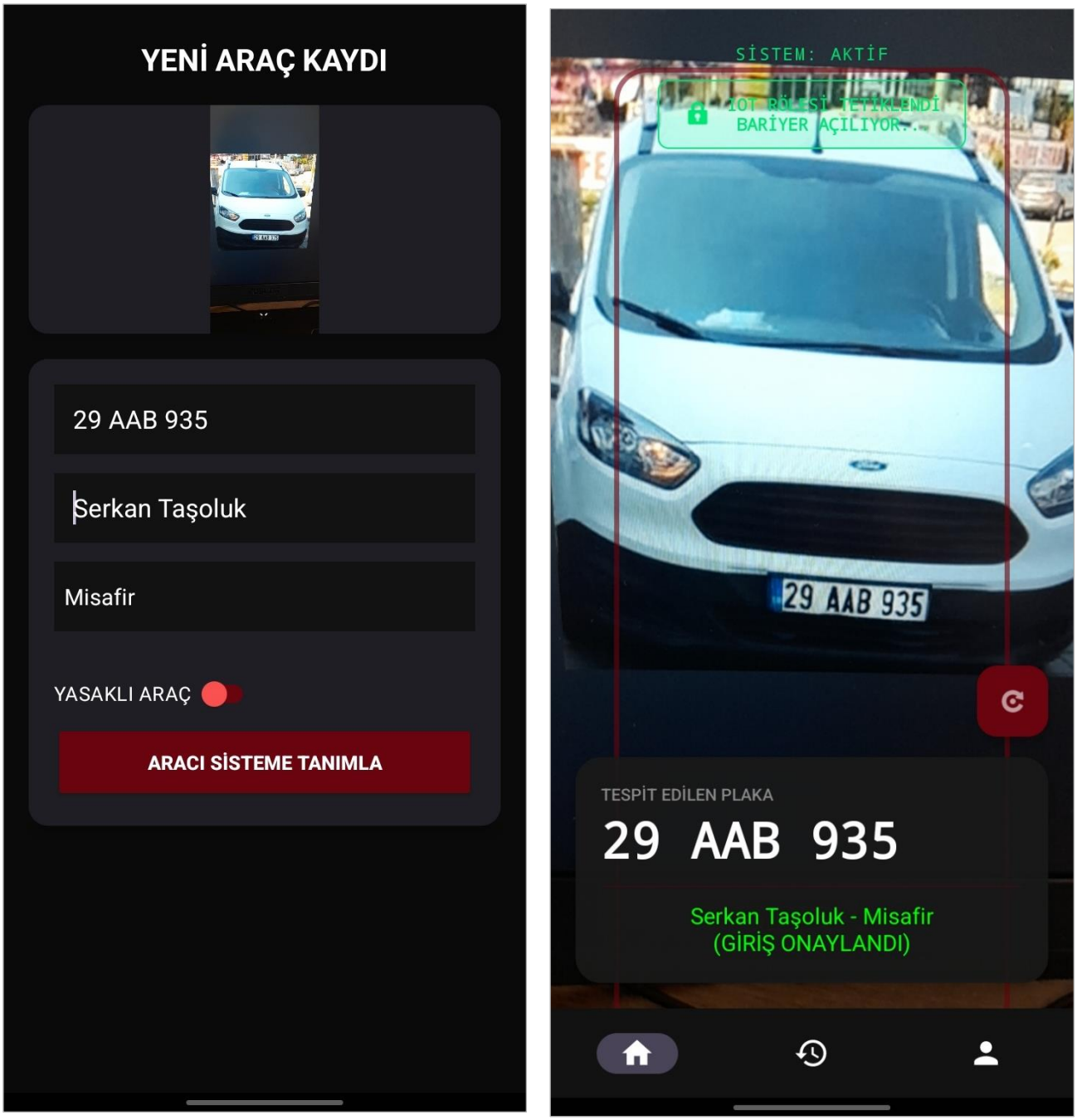


Şekil 10: Plaka Okuma ve Doğrulama Modülü

4.4.3. Sistem Yönetim Modülleri (Araç Ekleme, Personel Ekleme)

Veritabanına manuel veri girişinin yapıldığı idari modüllerdir (Örn: arac_ekleme.java). Kayıtsız (yabancı) bir araç nizamiyeye geldiğinde Dashboard modülünden otonom olarak bu modüle yönlendirme yapılır. Modül içerisinde yer alan form alanlarındaki girdiler kontrol edilir (Validation) ve sunucunun /api/arac-ekle uç noktasına iletilir.

İşlem sonucuna göre arayüzde başarılı/başarısız bildirimleri (Toast Message) gösterilir. Rol Tabanlı Erişim Kontrolü (RBAC) gereği, Personel Ekleme gibi alt modüllere sadece "Admin" yetkisiyle girildiğinde erişilebilir.

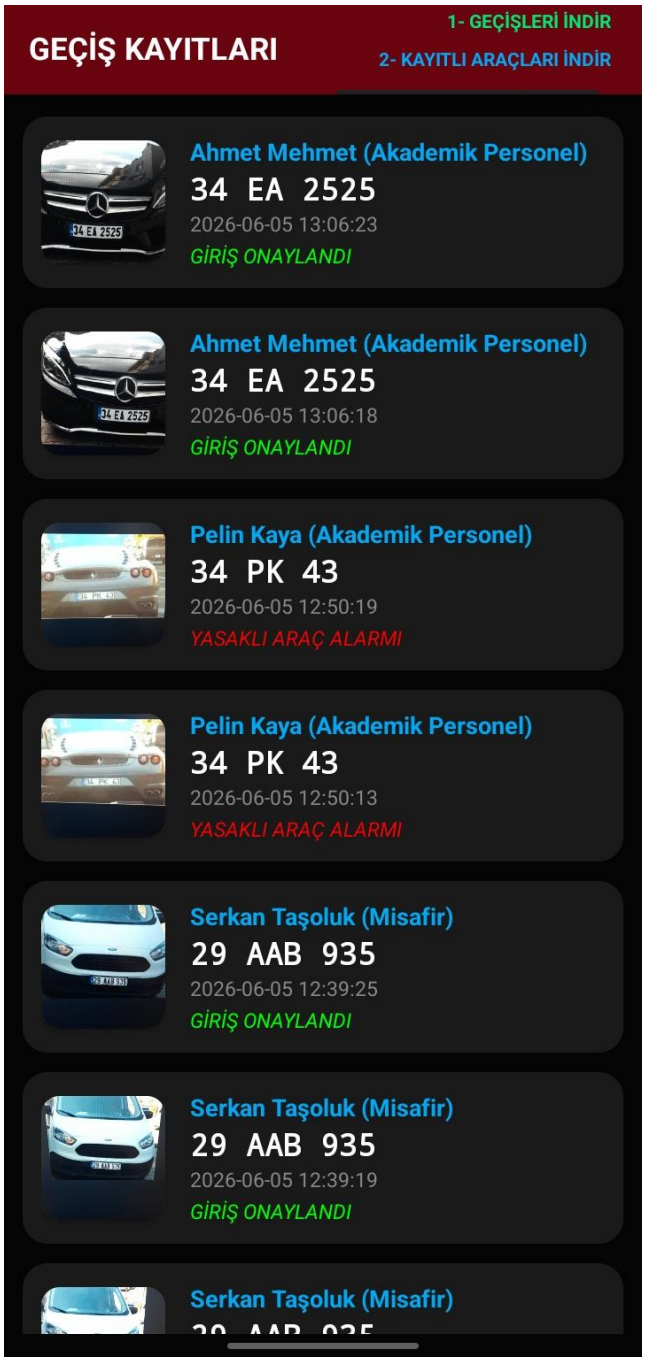


Şekil 11: Sistem Yönetim Modülleri

4.4.4. Raporlama Modülü

Geçmişe dönük geçiş loglarının dijital dokümanlara dönüştürüldüğü modüldür. Sunucudan çekilen son log kayıtları (JSON Array dizisi), Android platformunun yerel PdfDocument kütüphanesi kullanılarak işlenir. Sanal bir kanvas üzerinde tablolar, başlıklar ve log verileri sayfa koordinatlarına göre X ve Y eksenlerinde çizilir.

Oluşturulan bu sanal doküman, mobil cihazın dış depolama birimindeki (Downloads veya İndirilenler klasörü) fiziksel bir .pdf dosyasına yazılarak kurum idaresine sunulmaya hazır hale getirilir.



Şekil 12: PDF Raporlama Modülü Ekran Görüntüsü

5.SİSTEM GERÇEKLEŞTİRİMİ

5.1. Giriş

Sistem gerçekleştirim evresi, tasarım kararlarının programlama dilleri kullanılarak kaynak kodlara döküldüğü ve yazılımın hayat bulduğu süreçtir. Güvenli, bakımı kolay ve okunabilir bir yazılım oluşturmak amacıyla belirli kodlama standartları ve hata yakalama (Exception Handling) politikaları uygulanmıştır.

5.2. Yazılım Geliştirme Ortamları ve Diller

5.2.1. Programlama Dilleri ve Teknolojiler

Mobil uygulama, nesne yönelimli yapısı ve donanım kontrol yetenekleri nedeniyle Java dili ile Android Studio ortamında geliştirilmiştir. Sunucu (API) yazılımı ise hızlı prototipleme ve esnek veri işleme avantajlarından dolayı Python 3 ve Flask çatısı kullanılarak kodlanmıştır.

5.2.2. Veritabanı Yönetim Sistemleri (VTYS) ve SQLite Mimarisi

Sistemde RDBMS (İlişkisel Veritabanı Yönetim Sistemi) olarak **SQLite** tercih edilmiştir. SQLite'ın en büyük avantajı, bağımsız bir sunucu süreci (background process) gerektirmemesi ve tüm veritabanının tek bir disk dosyası (plaka_sistemi.db) içinde tutulabilmesidir.

Veri modeli; *Kullanicilar*, *Araclar* ve *Gecis_Kayitlari* olmak üzere 3 temel şemadan oluşur. Birincil anahtarlar (Primary Keys - ID) otomatik artan (AUTOINCREMENT) yapıdadır ve ilişkiler bu anahtarlar üzerinden sağlanır.

5.3. Kodlama Stili ve Yapısal Standartlar

5.3.1. Açıklama Satırları ve Kod Biçimlemesi

Programların okunabilirliğini artırmak için Java'da Javadoc standartları, Python'da ise Docstring ve inline yorum satırları kullanılmıştır. Girintileme (Indentation) kurallarına katı bir şekilde uyularak, her mantıksal blok (If, While, For) belirginleştirilmiştir.

5.3.2. Anlamlı İsimlendirme

Değişkenlerin ve metotların isimleri görevlerini yansıtacak şekilde seçilmiştir. Java modüllerinde *CamelCase* (Örn: ekranVeKilidiSifirla, plakaSayaci) formatı tercih edilirken, Python ortamında *Snake_Case* (Örn: kullanici_ekle, plaka_sorgula_api) standartları uygulanmıştır. Sihirli sayılar yerine sabitler kullanılmıştır.

5.4. Program Karmaşıklığı

Yazılımın döngüsel karmaşıklığını azaltmak için karmaşık Spagetti kod yapılarından kaçınılmıştır. Özellikle ML Kit'ten gelen anlamsız metinlerin elenmesi işleminde derin If-Else blokları yerine, güçlü bir matematiksel model olan düzenli ifadeler (Regex) örüntüsü kullanılarak karmaşıklık minimuma indirilmiş ve işlemci üzerindeki karar düğümleri azaltılmıştır.

5.4.1. McCabe Karmaşıklık Ölçütü Hesaplama

Plaka okuma ve doğrulama algoritmasının kontrol akış çizgesi çıkarıldığında; düğüm sayısı (N) 12, ayrıt (kenar) sayısı (E) 15 ve bağlı bileşen sayısı (P) 1 olarak tespit edilmiştir. Sistemin McCabe Döngüsel Karmaşıklık değeri;

$$V(G) = E - N + 2P$$

formülü ile hesaplanmıştır. $V(G) = 15 - 12 + 2(1) = 5$ Çıkan sonucun 10'dan küçük olması ($V(G) = 5$), yazılımın karmaşıklığının düşük seviyede olduğunu, test edilebilirliğinin ve bakımının oldukça kolay olduğunu matematiksel olarak kanıtlamaktadır.

5.5. Olağan Dışı Durum Çözümleme

5.5.1. Olağandışı Durum Tanımları ve Yönetimi

Gerçek zamanlı kamera akışı ve sunucu iletişiminde öngörülemeyen hataların sistemi çökertmemesi için try-catch (Java) ve try-except (Python) mekanizmaları kusursuz bir şekilde entegre edilmiştir:

- **Ağ (Volley) Kopmaları:** Mobil cihaz sunucuya erişemediğinde sistem "Anında Durdurma" yapmaz; bunun yerine *TimeoutError* veya *NetworkResponseError* fırlatılarak kullanıcıya "Sunucuya Ulaşılamadı" mesajı verilir ve kamera tarama döngüsü sıfırlanarak tekrar aktif edilir.
- **JSON Ayrıştırma Hataları:** Sunucudan hatalı formatta dönen cevaplarda oluşabilecek *JSONException* durumları yakalanır ve arayüz güvenli moda geçer.
- **Veritabanı Çakışmaları:** Sisteme zaten kayıtlı olan bir aracın tekrar kaydedilmek istenmesi durumunda Python tarafında *sqlite3.IntegrityError* hatası tetiklenir. Sunucu çökmez, bunun yerine istemciye "Bu plaka zaten sistemde var!" HTTP 400 hata kodu olarak nazikçe döndürülür.
- **Kriptografik Güvenlik:** Tüm yönetici ve görevli şifreleri veritabanına kaydedilirken *Werkzeug.security* kütüphanesinin Hash fonksiyonlarından geçirilir, böylece sistemin ele geçirilmesi durumunda bile parolalar şifreli kalır.

6.DOĞRULAMA VE GEÇERLEME

6.1. Giriş

Yazılım mühendisliği disiplinlerinde bir projenin kodlanması, o projenin tamamlandığı anlamına gelmez. Geliştirilen sistemin tasarımsal gereksinimleri tam olarak karşılayıp karşılamadığının kanıtlanması "Doğrulama", son kullanıcının (güvenlik görevlileri ve kurum idaresi) sahadaki operasyonel ihtiyaçlarını ve beklentilerini ne ölçüde giderdiğinin kanıtlanması ise "Geçerleme" olarak adlandırılır.

Fırat PTS projesi; donanım seviyesindeki mobil kamera lensleri, yapay zeka destekli optik karakter tanıma (OCR) motoru ve uzak sunucu tabanlı asenkron veri işleme mimarilerini bir araya getiren melez bir otomasyondur. Bu karmaşık yapı, sistemin çevresel faktörlere (ışık, ağ bağlantısı vb.) karşı duyarlı olmasını gerektirir. Bu nedenle, yazılımın yaşam döngüsü boyunca sistemin çökmesini engellemek ve %99'un üzerinde doğru okuma oranına ulaşmak amacıyla katı bir yazılım sınama (test) politikası yürütülmüştür.

6.2. Sınama Yöntemleri (Kara Kutu ve Beyaz Kutu Testleri)

Projenin test aşamalarında uluslararası yazılım test standartlarına uygun olarak iki temel yaklaşım benimsenmiştir:

- **Beyaz Kutu Sınaması (White Box Testing):** Bu yöntem, yazılımın iç mantığının, veri yapılarının ve algoritmik döngülerinin doğrudan kaynak kod üzerinden test edilmesidir. Projede özellikle makine öğrenmesi motorundan (Google ML Kit) dönen ham metinlerin süzüldüğü Düzenli İfadeler (Regex) algoritması Beyaz Kutu testine tabi tutulmuştur. Kod içerisindeki "if-continue" döngüleri ve karar noktaları sınanmıştır.
  Örneğin; arayüzde yer alan "SISTEM", "AKTIF" gibi kelimelerin veya 2 karakterden kısa anlamsız blokların, ağ üzerinden sunucuya gönderilmeden önce yerel cihazda algoritmik olarak nasıl bertaraf edildiği, koşul-yol (path coverage) testleriyle doğrulanmıştır.

- **Kara Kutu Sınaması (Black Box Testing):** Bu yöntemde kaynak kodun iç işleyişi bilinmeksizin, sistemin girdilere karşı verdiği çıktılar test edilir. Sistemin "Girdi" noktasını fiziksel bir araç plakası, "Çıktı" noktasını ise mobil cihaz ekranındaki renk değişimleri (kırmızı/yeşil) ve sesli bildirimler oluşturur. Kullanıcı arayüzünün (UI) beklenen tepkileri verip vermediği, sistemin sadece dış davranışları gözlemlenerek sınanmıştır.

6.3. Birim (Unit) ve Tümleştirme (Integration) Testleri

Test süreci, en küçük yapı taşından başlayarak sistemin geneline doğru genişleyen "Aşağıdan Yukarıya (Bottom-Up)" stratejisiyle yürütülmüştür.

- **Birim (Unit) Testleri:** Sistemdeki her bir fonksiyonun bağımsız olarak sınanmasıdır. Örneğin; Android cihazda yakalanan Bitmap görselin Base64 formatına çevrilmesi (kodlanması) işleminin kayıpsız yapılıp yapılmadığı veya Python sunucusunda `generate_password_hash` yordamıyla şifrelerin doğru formatta özetlenip özetlenmediği modüler olarak test edilmiştir.
- **Tümleştirme (Integration) Testleri:** Birim testlerinden geçen modüllerin birbiriyle olan iletişiminin sınanmasıdır. Öncelikle Python (Flask) sunucusundaki API uç noktaları (`/api/plaka-sorgula`) Postman aracı ile sahte JSON paketleri gönderilerek test edilmiştir. Sunucu tarafının çalıştığı doğrulandıktan sonra, Android tarafındaki Volley HTTP kütüphanesi bu uç noktalara entegre edilmiş ve mobil cihaz ile veritabanı (SQLite) arasındaki asenkron veri akışının bütünlüğü başarıyla doğrulanmıştır.

6.4. Sistem Test Senaryoları Matrisi

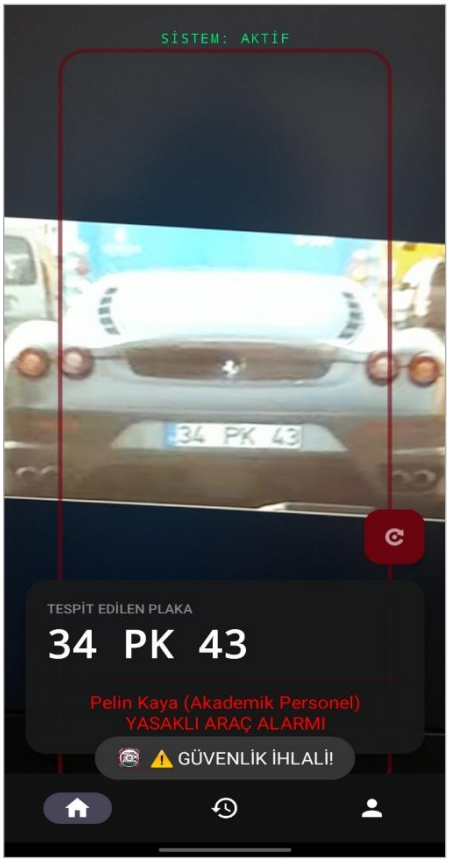
Sistemin saha şartlarındaki (Production) güvenilirliğini kanıtlamak amacıyla, Fırat Üniversitesi kampüs girişlerindeki fiziksel koşullar simüle edilerek aşağıdaki uçtan uca sistem test senaryoları tasarlanmış ve koşturulmuştur.

6.4.1. Gündüz ve İdeal Koşullar Testi

- **Ön Koşul:** Havanın aydınlık olduğu, plakanın temiz ve okunabilir olduğu standart dış ortam.
- **Eylem:** Veritabanındaki ARACLAR tablosunda kaydı bulunan ve yetki statüsü "ONAYLI" olan akademik/idari bir personelin aracı mobil kameraya okutulur.
- **Beklenen Sonuç:** Sistem plakayı Regex süzgecinden geçirerek sunucuya iletir. Sunucudaki SQL sorgusu olumlu döner (YESIL_ISIK). Arayüz yeşil renge bürünür ve görevliye geçişin onaylandığına dair standart bir onay sesi (Beep) iletilir. Toplam tepki süresinin 1.5 saniyenin altında olması beklenir.
- **Gerçekleşen Durum:** İşlem 1 saniyenin altında başarıyla sonuçlanmıştır. (BAŞARILI).

6.4.2. Yasaklı Araç (Kırmızı Alarm) ve İhlal Tespiti Testi

- **Ön Koşul:** Disiplin veya güvenlik sebebiyle kampüse girişi engellenmiş bir aracın veritabanında "YASAKLI" olarak etiketlenmiş olması.
- **Eylem:** Yasaklı aracın plakası mobil cihaz tarafından taranarak sunucuya gönderilir.
- **Beklenen Sonuç:** Sunucudaki sorgu neticesinde durum kodu "KIRMIZI_ALARM" olarak JSON formatında istemciye döner. Görevlinin dikkatini anında çekmek için cihaz ekranı tamamen kırmızıya boyanır. Donanımsal olarak Acil Durum Sesi çalınır ve cihazın Titreşim (Vibrator) motoru en yüksek seviyede tetiklenir.
- **Gerçekleşen Durum:** Görsel ve işitsel alarmlar senkronize şekilde devreye girmiş, güvenlik ihlali loglanmıştır. (BAŞARILI).



Şekil 13: Gündüz ve İdeal Koşullar Test Sonucu Ekranı

6.4.3. Kayıtsız (Yabancı) Araç Tespiti Testi

- **Ön Koşul:** Ziyaretçi veya kargo firması gibi sistemde hiçbir kaydı bulunmayan yabancı bir aracın nizamiyeye gelmesi.
- **Eylem:** Görevli plakayı okutur ve veritabanı taraması başlar.
- **Beklenen Sonuç:** Veritabanından hiçbir eşleşme bulunamaz. Sistem "YABANCI" durum kodunu döndürerek ekranı uyarı moduna (sarı/turuncu) geçirir. Görevlinin ekranında "Araç Ekle" butonu otomatik olarak aktifleşir ve ilgili verilerle beraber kayıt ekranına yönlendirme yapılır.
- **Gerçekleşen Durum:** Yabancı araç doğru tespit edilmiş ve dinamik menü yönetimi sayesinde görevli kayıt formuna başarıyla aktarılmıştır. (BAŞARILI).

6.4.4. Peş Peşe Geçiş (Tailgating / Flood) Engelleme Testi

- **Ön Koşul:** Kameranın görüş açısında sabit duran veya birbirini çok yakından takip eden araçların bulunması.
- **Eylem:** Kamera saniyede 30 kare (FPS) tarama yaptığı için, aynı plakanın defalarca okunup sunucuya saniyede 30 adet HTTP POST isteği gönderilmesi (DDoS / Flood etkisi) simüle edilir.
- **Beklenen Sonuç:** Yazılım mimarisinde tasarlanan "Kilitlenme ve Sayaç" algoritması devreye girmelidir. Aynı plaka ardışık olarak en az 3 kez tespit edilmeden (onay_siniri = 3) sunucuya istek atılmamalıdır.

  Hedef kilitlendikten sonra istek sadece bir kez atılmalı ve sistem kendini 3 saniye boyunca uykuya alarak arka arkaya spam gönderimini engellemelidir.

- **Gerçekleşen Durum:** İstemci (Android) tarafındaki mantıksal sayaç sorunsuz çalışmış, sunucu gereksiz ağ trafiğinden (Network Overhead) korunmuştur. (BAŞARILI).

6.4.5. Çevrimdışı Kalma ve Ağ Kopması Testi

- **Ön Koşul:** Nizamiyedeki Wi-Fi veya hücresel ağ bağlantısının zayıf olması veya tamamen kopması.
- **Eylem:** Mobil cihaz plakayı tam olarak okuyup, fotoğrafı Base64'e çevirip sunucuya HTTP isteği gönderdiği (Volley tetiklendiği) milisaniyede internet bağlantısı manuel olarak kesilir.
- **Beklenen Sonuç:** Uygulamanın kesinlikle çökmemesi (Crash / ANR olmaması) gerekir. Ağ isteğini yöneten kütüphanenin "Zaman Aşımı (TimeoutError)" veya "Ağ Hatası (NetworkError)" fırlatması ve `try-catch` bloklarının bu hatayı yakalaması beklenir. Arayüzde kırmızı renkte "Bağlantı Hatası! Sunucuya ulaşılamadı" mesajı basılmalı ve sistem tarama mekanizmasını sıfırlayarak çökmeyi önlemelidir.
- **Gerçekleşen Durum:** Hata yönetimi (Exception Handling) mekanizması devreye girmiş, uygulama çökmeden bağlantı uyarısını güvenli bir şekilde operatöre iletmiştir. (BAŞARILI).

6.5. Kalite Güvence ve Sistem Onay Matrisi (QA Matrix)

Sahada gerçekleştirilen tüm senaryo testlerinin sonuçları aşağıdaki Kalite Güvence Onay Matrisinde özetlenmiştir.

Tablo 6: Sistem Sınama ve Onay Matrisi

Test Kodu	Test Modülü / Senaryosu	Beklenen Sistem Davranışı	Durum
TST-01	OCR Algoritma Doğruluğu	ML Kit'in plakayı doğru string değerine çevirmesi.	**[GEÇTİ]**
TST-02	Regex Metin Filtreleme	Sadece Türkiye plaka formatına uyan metinlerin seçilmesi.	**[GEÇTİ]**

Test Kodu	Test Modülü / Senaryosu	Beklenen Sistem Davranışı	Durum
TST-03	Onaylı Araç Geçişi	1.5 sn içinde yeşil ekran ve onay sesi verilmesi.	**[GEÇTİ]**
TST-04	Yasaklı Araç (Kırmızı Alarm)	Ekranın kırmızı olması, yüksek titreşim ve acil durum sesi.	**[GEÇTİ]**
TST-05	Ardışık Okuma (Flood) Koruması	3 kez okumadan kilitlenmemesi ve sunucunun korunması.	**[GEÇTİ]**
TST-06	Ağ Kopması (Exception Handling)	Uygulamanın çökmemesi ve "Bağlantı Hatası" uyarısı vermesi.	**[GEÇTİ]**

7.BAKIM, KURULUM VE SÜRDÜRÜLEBİLİRLİK

7.1. Giriş

Yazılım sistemleri, donanımlar gibi fiziksel ve mekanik olarak aşınmaya uğramasalar da zaman içerisinde çevre koşullarının değişmesi, veri tabanı hacminin geometrik olarak büyümesi, işletim sistemi güncellemeleri ve kullanıcı ihtiyaçlarının evrilmesi nedeniyle sürekli bir idame ve optimizasyon döngüsüne ihtiyaç duyarlar.

Fırat Üniversitesi merkez kampüs nizamiyelerinde 7/24 esasına göre kesintisiz hizmet vermesi planlanan Fırat PTS (Plaka Tanıma Sistemi) projesinin başarısı, sadece ilk üretim aşamasına değil, sistemin yıllar boyu kararlı çalışmasını sağlayacak olan sürdürülebilirlik kriterlerine de doğrudan bağlıdır.

Yazılım mühendisliği yaşam döngüsünün (SDLC) en uzun ve maliyetli evresini oluşturan bu aşamada; yazılımın canlı ortama hatasız aktarılması, sunucu kaynaklarının verimli kullanılması ve veri tabanı performansının ilk günkü milisaniyeler seviyesinde korunması amacıyla uygulanacak stratejik süreçler planlanmıştır.

7.2. Sistemin Canlı Ortama Kurulumu

Geliştirme (Development) ortamında lokal ağ ve emülatörler üzerinde test edilen sistemin, sahada gerçek operasyonu yürütecek olan fiziksel cihazlara ve sunuculara aktarılması işlemi "Canlı Ortama Kurulum (Deployment)" sürecidir. Fırat PTS dağıtık bir istemci-sunucu mimarisine sahip olduğundan, kurulum adımları eşgüdümlü olarak iki farklı katmanda yürütülür:

- **Sunucu Katmanının Canlıya Alınması:** Kampüs bilgi işlem merkezinde tahsis edilen ana sunucu bilgisayarı üzerinde kararlı bir Python 3.x çalışma ortamı yapılandırılır.

  Projenin bağımlılıkları olan Flask mikro web çatısı, Werkzeug güvenlik kütüphanesi ve pytz zaman kalibrasyon modülleri sunucuya yüklenir. Geliştirme aşamasında lokal ağ kısıtlamalarını aşmak için kullanılan geçici Ngrok tüneli yerine, nizamiyedeki mobil terminallerin her an sabit bir adresten erişebilmesi için sunucuya kurumsal bir Statik IP adresi atanır.

  Güvenlik duvarı (Firewall) üzerinden Flask uygulamasının yayın yaptığı port (Varsayılan olarak Port 5000) dış isteklere güvenli bir şekilde açılır. İlişkisel veritabanı kütüğü olan plaka_sistemi.db dosyası ilk kez çalıştırılarak tabloların şemaları disk üzerinde oluşturulur.

- **Mobil İstemci Katmanının Dağıtımı:** Android Studio ortamında nesne yönelimli programlama standartlarında kodlanan istemci yazılımı, projenin canlı sunucu Statik IP adresini barındıracak şekilde ağ konfigürasyon dosyalarından güncellenir.

  Uygulama, Google Play Store yükleme standartlarına veya kurumsal yerel dağıtıma uygun şekilde, kurumsal güvenlik (Release) anahtarlarıyla imzalanarak tescilli bir .apk (Android Package Kit) paketi haline derlenir. Bu paket, kampüs girişindeki görevlilerin el terminallerine veya sabit mobil cihazlarına yüklenir.

  İlk çalıştırma esnasında, Android işletim sisteminin modern güvenlik protokolleri gereği uygulamanın "Donanımsal Kamera" (CameraX görüntü akışı için) ve "Dış Depolama Birimi" (Geçiş raporlarının PDF olarak yazdırılabilmesi için) erişim izinleri operatör tarafından kalıcı olarak onaylanır.

7.3. Yazılım Bakımı ve Optimizasyon

Canlı ortama alınan Fırat PTS yazılımı, her gün kampüse giriş ve çıkış yapan binlerce aracın verisini işlemekle yükümlüdür. Bu durum, zamanla sistem kaynaklarının (bellek, işlemci, depolama alanı) optimizasyonunu zorunlu kılar. Yazılım mühendisliği standartlarına uygun olarak sistem üzerinde üç temel bakım faaliyeti yürütülür:

Düzeltici bakım (Corrective Maintenance) ile sahada ortaya çıkabilecek öngörülemeyen istisnai ağ hataları loglanarak yamalarla onarılır. Uyarlayıcı bakım (Adaptive Maintenance) ile ilerleyen dönemde Android işletim sisteminin veya Google ML Kit yapay zeka motorunun alabileceği büyük mimari güncellemelerin kod katmanına entegrasyonu sağlanır. En kritik süreç olan önleyici ve mükemmelleştirici bakım ise projenin donanım limitlerini zorlamasını engelleyen veri optimizasyonu faaliyetlerini kapsar.

7.3.1. Biriken Görsel Logların Arşivlenmesi ve Silinmesi

Fırat PTS projesinin en ayırt edici mimari özelliklerinden biri, fotoğrafların veritabanını şişirmesini önlemek amacıyla uygulanan hibrit depolama yaklaşımıdır. Android terminal, plakasını sabitlediği aracın o anki fotoğraf karesini Base64 formatında metne dönüştürerek sunucuya iletir. Sunucu ise bu metni çözerek (decode) disk üzerinde fiziksel bir .jpg dosyası olarak saklar ve veritabanına sadece bu dosyanın yolunu (path) kaydeder.

Ancak kampüs trafiği göz önüne alındığında, aylık bazda on binlerce, yıllık bazda ise yüz binlerce yüksek çözünürlüklü araç fotoğrafı sunucu diskinde (Hard Disk / SSD) depolanacaktır. Bu durum, zamanla sunucuda ciddi bir depolama alanı darboğazına yol açma riski taşır.

Bu riski proaktif olarak yönetmek adına sistem üzerinde "Görsel Log Yönetim Politikası" tanımlanmıştır. Sistem yöneticisi (Admin) paneline entegre edilecek veya sunucu işletim sisteminde otomatik zamanlanmış görevlerle çalıştırılacak bir optimizasyon scripti ile, 6 aydan daha eski geçiş kayıtlarına ait fotoğraflar tespit edilir.

Bu fotoğraflar, sunucunun ana diskini meşgul etmemesi amacıyla harici bir bulut depolama alanına veya yedekleme sunucusuna (NAS cihazlarına) adli bilişim standartlarında tarih bazlı klasörlenerek arşivlenir (Cold Storage). Arşivleme işlemi doğrulandıktan sonra, sunucu diskindeki yerel fotoğraflar otonom olarak silinerek sistemin depolama alanı sürekli olarak açık ve optimize tutulur.

7.3.2. SQLite Veritabanı Bakımı (Vacuuming)

Sistemde kullanılan ilişkisel veritabanı yönetim sistemi (RDBMS) olan SQLite, veri saklama teknolojisi olarak son derece hafif ve hızlıdır. Ancak SQLite mimarisinin depolama motorunda çok önemli bir teknik detay bulunmaktadır:

Bir tablodan (Örn: gecis_kayitlari tablosundan) veri silindiğinde veya eski log kayıtları temizlendiğinde, SQLite bu verilerin kapladığı disk alanını işletim sistemine o anda iade etmez. Silinen satırların yerini "Boş Sayfalar (Free Pages)" olarak işaretler ve veritabanı dosyasının fiziksel boyutu (kütük boyutu) disk üzerinde aynı kalır.

Zamanla binlerce kaydın silinmesi veya güncellenmesi sonucunda veritabanı dosyasında ciddi bir içsel parçalanma (Internal Fragmentation) meydana gelir. Bu durum, yeni kayıtlar eklenirken veya geçmiş loglar arasında arama yapılırken B-Tree indeksleme algoritmalarının yavaşlamasına, dolayısıyla sistemin performansının düşmesine neden olur.

Fırat PTS projesinin sürdürülebilirlik planı kapsamında bu sorunu çözmek amacıyla *SQLite Vacuuming (Veritabanı Sıkıştırma)* optimizasyonu uygulanacaktır.

Periyodik bakım dönemlerinde (Örn: her ayın sonunda veya görsel loglar arşivlendikten hemen sonra) sunucu katmanında SQL motoru üzerinden VACUUM komutu tetiklenecektir. Bu komut arka planda;

- Parçalanmış durumdaki veritabanı kütüğünü tamamen tarar,
- İçerisindeki boş sayfaları ve silinmiş satırların bıraktığı artıkları temizler,
- Tablo indekslerini doğrusal bir sırada yeniden inşa eder (Defragmentation),
- Veritabanı dosyasını disk üzerinde ardışık bloklar halinde yeniden yazarak fiziksel dosya boyutunu küçültür.

Bu mükemmelleştirici bakım adımı sayesinde, gecis_kayitlari tablosu milyonlarca satıra ulaşsa dahi, plaka sorgulama ve yetki doğrulama isteklerine sunucunun verdiği yanıt süresi (Response Time) her zaman projenin başarım kriteri olan 1.5 saniyenin altında kalacaktır.

8.PROJEDE ÖNGÖRÜLEN EKSİKLİKLER VE GELECEK ÇALIŞMALAR

8.1. Mevcut Sürümde Eksik Kalan Modüller ve Teknik Gerekçeleri

Yazılım mühendisliği projelerinde sistem tasarımı ile saha uygulaması arasında çeşitli çevresel, donanımsal ve idari kısıtlamalar nedeniyle farklılıklar doğabilmektedir. Literatürde "Kapsam Kayması (Scope Creep)" veya donanımsal darboğazlar olarak adlandırılan bu durumlar, projenin bazı modüllerinin sonraki sürümlere (v2.0) ertelenmesine neden olabilir.

Projenin en büyük **artısı ve tercih edilme sebebi**; yüz binlerce liralık endüstriyel donanımlara (özel ANPR kameralarına ve yüksek işlem gücüne sahip devasa sunuculara) ihtiyaç duymaksızın, bir güvenlik görevlisinin cebindeki standart akıllı telefonu devasa bir güvenlik ekosisteminin merkezine oturtmasıdır.

Görüntü işleme yükünün buluta veya sunucuya gönderilmeden doğrudan cihaz üzerinde (Uç Bilişim - Edge Computing) yapılması ağ bant genişliği tüketimini neredeyse sıfıra indirmiştir. Sunucu tarafında ise hantal veritabanı yönetim sistemleri (SQL Server, Oracle vb.) yerine hafif siklet bir SQLite veritabanı ve Flask web çatısının kullanılması, projenin kaynak tüketimini (Resource Footprint) minimuma indirerek sistemin her türlü donanımda koşturulabilmesini sağlamıştır.

Sistemin manuel kağıt kayıt defterlerinin yerini alması, hem geçmişe dönük geçiş loglarının anında PDF olarak raporlanabilmesini sağlamış hem de Kişisel Verilerin Korunması Kanunu (KVKK) ihlali risklerini ortadan kaldırmıştır.

Fırat Üniversitesi kampüs nizamiyeleri için tasarlanan PTS projesinde, başlangıçta hedeflenen ancak mevcut sürümde tam olarak devreye alınamayan temel modüller ve bunların eksik kalma gerekçeleri aşağıda detaylandırılmıştır:

- **Fiziksel Bariyerin Otonom Tetiklenmesi (IoT Entegrasyonu):** Projenin orijinal mimari kurgusunda, mobil uygulamadan sunucuya giden plaka verisi sonucunda "GİRİŞ ONAYLANDI" (HTTP 200 OK) yanıtı alındığında, sistemin hiçbir insan müdahalesine gerek kalmadan nizamiye bariyerini otomatik açması planlanmıştı.

  Bu otonom yapının kurulabilmesi için mobil cihazın veya Python sunucusunun, bariyer motoruna bağlı bir mikrodenetleyici ve bir röle kartı ile yerel ağ üzerinden TCP/IP soketleri veya MQTT protokolü aracılığıyla haberleşmesi gerekmekteydi. Ancak geliştirme sürecinde, bariyerin altında araç varken kapanmasını engelleyecek fiziksel güvenlik sensörlerinin (Loop dedektörleri veya kızılötesi bariyer sensörleri) eksikliği tespit edilmiştir.

  Literatürde "Fail-Safe (Hata Durumunda Güvenli Kalma)" prensibi gereği, sensörsüz bir otonom bariyerin araçlara veya yayalara fiziksel zarar verme riski bulunduğundan ve kampüs ağındaki katı güvenlik duvarı (Firewall) yapılandırmalarının yerel soket iletişimini kısıtlamasından dolayı bu IoT entegrasyonu ertelenmiştir.

  Şu anki yapıda sistem "yarı-otonom" çalışmakta; yazılım onay ve ret kararlarını saniyeler içinde hatasız vermekte, bariyerin açılma işlemi ise görsel onaydan sonra güvenlik görevlisinin manuel inisiyatifinde (buton aracılığıyla) yürütülmektedir.

- **Zorlu İklim Koşulları İçin Görüntü İyileştirme (Image Pre-processing) Modülü:** Elazığ ilinin zorlu kış şartlarında, özellikle plakaların kar, buz veya çamurla kapandığı durumlarda ya da gece yoğun far parlamalarında standart mobil kameraların Optik Karakter Tanıma (OCR) başarı oranı düşebilmektedir.

  Görüntüyü Google ML Kit motoruna vermeden önce parlamayı azaltacak (WDR simülasyonu), kontrastı artıracak ve bulanıklığı giderecek özel bir görüntü ön-işleme algoritmasının sisteme dahil edilmesi planlanmıştır. Ancak bu tür piksel tabanlı matris işlemlerinin mobil cihazın CPU ve GPU birimlerine aşırı yük bindirdiği, pil tüketimini dramatik ölçüde artırdığı ve 1.5 saniyelik yanıt süresi (Response Time) hedefini aştığı gözlemlendiği için bu modül şimdilik kapsam dışı bırakılmıştır.

8.1.Projeye İleride Eklenebilecek Özellikler

Projenin yazılım mimarisi, Nesne Yönelimli Programlama (OOP) ilkelerine uygun olarak yeni eklentilere (Plug-in) ve modüler büyümeye açık (Scalable) bir yapıda tasarlanmıştır. Gelecek çalışmalarda sistemi endüstri standartlarının ötesine taşıyacak eklenti planları şunlardır:

- **Derin Öğrenme (Deep Learning) ile Çift Faktörlü Doğrulama:** Mevcut sistem plaka üzerindeki metinleri okuyarak doğrulama yapmaktadır.

  Araç plakalarının sökülüp başka bir araca takılması (sahte plaka / ikiz plaka) gibi fiziksel güvenlik ihlallerini önlemek amacıyla sisteme Evrişimli Sinir Ağları (CNN) tabanlı YOLOv8 veya ResNet gibi gelişmiş nesne tanıma modelleri eklenebilir. Böylece plaka ile birlikte aracın "Marka, Kasa Tipi ve Renk" bilgisi de kameradan tespit edilerek, veritabanındaki tescil kaydıyla eşleştirilecek ve çift faktörlü tam güvenlik (Two-Factor Authentication) sağlanacaktır.

- **Bulut Tabanlı (Cloud) Veri Senkronizasyonu ve Ölçeklenebilirlik:** Mevcut lokal SQLite mimarisinden Firebase, Amazon Web Services (AWS) RDS veya PostgreSQL gibi bulut tabanlı, eşzamanlı ilişkisel veritabanlarına geçiş yapılması planlanmaktadır. Bu sayede üniversitenin tüm giriş kapılarının (Rektörlük, Teknokent, Araştırma Hastanesi, Eğitim Fakültesi vb.) verileri yatay olarak ölçeklenebilecek (Horizontal Scaling) ve eşzamanlı olarak tek bir merkezi veri havuzunda birleştirilecektir.

- **Web Tabanlı İdari Gösterge Paneli (Admin Dashboard):** Raporların yalnızca PDF olarak mobil cihazdan alınması yerine, sistem yöneticilerine ve rektörlük idari birimlerine özel, Python Flask API'si üzerine entegre edilmiş React.js veya Vue.js tabanlı modern bir Web Yönetim Paneli geliştirilebilir. Bu panelde JWT (JSON Web Token) tabanlı güvenli giriş yapılarak; kampüsün hangi saatlerde en yoğun trafiği yaşadığı, ısı haritaları (Heatmap), anlık log akışları (WebSocket ile canlı izleme) ve personel performans metrikleri grafiksel veri görselleştirme (Data Visualization) araçlarıyla sunulabilir.

8.2.Projenin Geliştirme Süresi Boyunca Bize Katkısı

Bu projenin planlama, çözümleme, tasarım, gerçekleştirim ve sınama aşamalarının baştan sona tek bir çatı altında "Tam Yığın (Full-Stack)" mimariyle yürütülmesi; teorik yazılım mühendisliği, proje yönetimi ve sistem tasarımı konseptlerinin gerçek dünya problemlerine uygulanması açısından eşsiz bir akademik kazanım sağlamıştır.

Geliştirme sürecinin şahsıma sağladığı temel mesleki ve akademik katkılar şunlardır:

- **Mobil Programlama ve Donanım Yönetimi:** Java dili ile Android yaşam döngüsüne (Lifecycle) hâkim olunmuş, cihaz kamerasından yüksek çözünürlüklü görüntü akışı alabilmek için CameraX kütüphanesi başarıyla entegre edilmiştir.
- **Yapay Zeka ve Optik Karakter Tanıma (OCR):** Google ML Kit kullanılarak makine öğrenmesi modellerinin mobil platformlarda çevrimdışı (offline) olarak nasıl koşturulacağı tecrübe edilmiştir.

- **API Tasarımı ve Ağ İletişimi:** Python Flask kullanılarak RESTful API uç noktalarının (`/api/plaka-sorgula`, `/api/giris`) nasıl yazılacağı, asenkron ağ isteklerinin Android Volley kütüphanesi ile nasıl yönetileceği ve Base64 ile şifrelenmiş görsel verilerin (JSON üzerinden) nasıl kayıpsız taşınacağı pratik edilmiştir.
- **Veri Mühendisliği ve Güvenlik:** Düzenli İfadeler (Regex) ile metin süzme, SQLite ile ilişkisel veritabanı kurma ve Werkzeug kütüphanesi ile kullanıcı parolalarını kriptografik olarak Hash'leme gibi siber güvenlik konseptleri projeye başarıyla uygulanmıştır.

Proje Yönetimi ve Sınama: İşlev Noktası ve COCOMO analizi yapılarak bir yazılımın maliyet ve satır (LOC) kestirimlerinin nasıl hesaplanacağı öğrenilmiş; Beyaz/Kara Kutu sınama yöntemleri ile yazılım kalitesinin nasıl artırılacağı uygulamalı olarak görmüş bulundum.

9.PROJE TESLİM KILAVUZU

Bir yazılım mühendisliği projesinin başarıyla tamamlanmış sayılabilmesi için, geliştirme ortamından (Development) bağımsız olarak, son kullanıcının veya kurum bilgi işlem birimlerinin sistemi kendi altyapılarına kolayca entegre edebilmeleri gerekmektedir.

Fırat PTS projesi dağıtık bir (İstemci-Sunucu) mimariye sahip olduğundan, teslim edilecek kaynak kodların ve derlenmiş paketlerin kurulum süreçleri belirli bir disiplin ve sıraya göre yapılmalıdır. Aşağıda sistemin sıfırdan ayağa kaldırılması için gereken teknik kılavuz adımları detaylandırılmıştır.

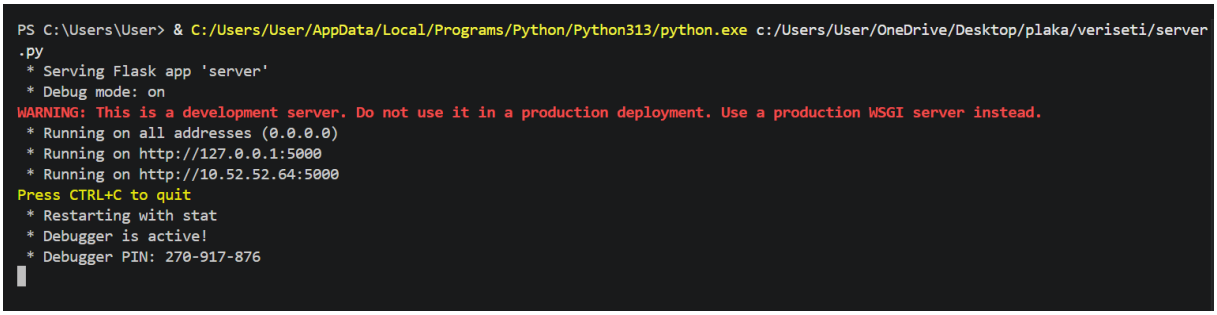
9.1. Python Sunucusunun Ayağa Kaldırılması

Sistemin beyni konumunda olan ve veritabanı işlemlerini yürüten Python Flask Backend sunucusunun çalıştırılması için izlenmesi gereken adımlar şunlardır:

Çalışma Ortamının Hazırlanması: Sunucu görevi görecek bilgisayara Python 3.10 veya üzeri bir sürüm yüklenmelidir. İşletim sistemindeki diğer Python projeleriyle kütüphane çakışmalarını önlemek amacıyla terminal (Komut İstemi) açılarak projenin bulunduğu dizinde python -m venv venv komutu ile izole bir sanal ortam (Virtual Environment) oluşturulur. Ardından Windows sistemler için venv\Scripts\activate komutu ile bu ortam aktifleştirilir.

1. **Gerekli Bağımlılıkların (Dependencies) Yüklenmesi:** Projenin sorunsuz çalışması için dış kütüphanelere ihtiyaç vardır. Terminal üzerinden pip install flask werkzeug pytz komutu çalıştırılarak REST API altyapısı için Flask, şifreleme ve güvenlik algoritmaları için Werkzeug ve zaman damgası kalibrasyonları için Pytz kütüphaneleri sisteme dahil edilir.

2. **Veritabanının İlklendirilmesi (Initialization):** Komut satırından python app.py komutu verilerek sunucu başlatılır. Sistem ilk kez çalıştığında, kod içerisindeki algoritmalar sayesinde otomatik olarak plaka_sistemi.db isimli SQLite kütüğü disk üzerinde oluşturulur ve gerekli tablolar (Kullanıcılar, Araçlar, Geçiş Kayıtları) şemaya uygun olarak inşa edilir. Sunucu, başarılı bir şekilde ayağa kalktığında terminalde Running on http://127.0.0.1:5000/ ibaresi belirecektir.



Şekil 14: Python Sunucusunun Başlatılması ve Veritabanı İlklendirme Akışı

9.2. Ngrok Bağlantısının Sağlanması ve API Adreslerinin Güncellenmesi Yerel ağda (Localhost) çalışan Python sunucusunun, dış dünyadaki (saha ortamındaki) Android mobil cihazlarla iletişim kurabilmesi için güvenli bir ağ tüneline ihtiyacı vardır. Geliştirme ve test süreçlerinde bu tünelleme işlemi için **Ngrok** aracı kullanılmıştır. Kurulumu şu şekilde yapılır:

1. Sunucu bilgisayarında Ngrok yazılımı başlatılarak terminal ekranına ngrok http 5000 komutu girilir.
2. Ngrok, yerel bilgisayardaki 5000 portunu dış dünyaya açarak HTTPS protokolüne sahip geçici ve güvenli bir yönlendirme URL'si üretir.
3. Elde edilen bu güvenli adres kopyalanır ve Android Studio projesi içerisindeki uç nokta (Endpoint) değişkenlerine yapıştırılır. Özellikle DashBoardActivity.java ve diğer sayfalardaki ağ isteklerini yöneten Volley kütüphanesi yapılandırmalarındaki sunucuURL ve girisurl değişkenleri bu yeni adres ile güncellenir. Bu adım, mobil cihazın çektiği fotoğrafların ve okuduğu plakaların doğru sunucuya ulaşması için kritik öneme sahiptir.

9.3. Android Uygulamasının Derlenip (APK) Cihaza Yüklenmesi Sistemin ön yüz (Frontend) ve yapay zeka (Edge AI) birimlerinin derlenerek son kullanıcının mobil donanımına aktarılma süreci (Deployment) aşağıdaki adımlarla tamamlanır:

1. **Derleme Öncesi İzin Kontrolleri:** Android Studio üzerinde proje açıldıktan sonra AndroidManifest.xml dosyası kontrol edilir. Sistemin çalışması için hayati önem taşıyan CAMERA (Kamera ile plaka okuma için), INTERNET (Sunucu iletişimi için) ve WRITE_EXTERNAL_STORAGE (PDF raporlarını cihaza kaydetmek için) izinlerinin manifest dosyasına eklendiği doğrulanır.
2. **APK İnşası (Build Process):** Menü çubuğundan *Build > Build Bundle(s) / APK(s) > Build APK(s)* yolu izlenerek Java kaynak kodları makine diline çevrilir. Derleyici, ML Kit ve CameraX kütüphanelerini de paketin içine entegre ederek app-debug.apk (veya imzalı app-release.apk) dosyasını oluşturur.

3. **Fiziksel Cihaza Kurulum:** Oluşturulan APK dosyası, bir USB kablosu aracılığıyla veya güvenli bir dosya aktarım servisi üzerinden kampüs nizamiyesinde kullanılacak olan Android işletim sistemli terminal cihazına kopyalanır. Cihaz üzerinden dosya açılarak kurulum işlemi (Install) başlatılır. Kurulum bittikten sonra uygulamanın kamera ve hafıza erişim izinleri cihaz ayarlarından aktif edilerek sistem operasyonel (kullanıma hazır) duruma getirilir.

10.SONUÇ VE DEĞERLENDİRME

10.1. Projenin Genel Değerlendirmesi (Artıları, Eksileri ve Tercih Sebepleri)

Fırat Üniversitesi merkez kampüs nizamiyelerindeki trafik darboğazını, manuel kağıt kayıt sistemlerinin hantallığını ve güvenlik zafiyetlerini çözmek amacıyla geliştirilen Plaka Tanıma Sistemi (PTS), yazılım mühendisliği prensiplerinin modern yapay zeka araçlarıyla (Google ML Kit) harmanlandığı, yenilikçi bir otomasyon çözümüdür.

Artıları ve Tercih Sebepleri: Projenin en büyük gücü; yüksek bütçeli, ağır donanımlara (özel ANPR kameralarına ve yüksek CPU gücüne sahip devasa fiziksel sunuculara) duyulan ihtiyacı ortadan kaldırmasıdır. Güvenlik görevlisinin standart akıllı telefonunu, karmaşık bir yapay zeka ve güvenlik ekosisteminin merkezine oturtmayı başarmıştır.

Görüntü işlemenin bulutta değil cihaz üzerinde (Edge Computing) yapılması, ağ bant genişliği tüketimini neredeyse sıfıra indirmiştir. Sunucu tarafında ise hantal ve lisans ücretli veritabanı yönetim sistemleri (Oracle, MSSQL vb.) yerine hafif siklet bir SQLite veritabanının ve Flask web çatısının kullanılması, projenin kaynak tüketimini (Resource Footprint) optimize etmiştir. Ayrıca kağıt kayıt defterlerinin yerini alarak geçiş loglarının anında PDF olarak dijital ortamda raporlanabilmesini sağlamış, geriye dönük sorgulamaları saniyelere düşürmüştür.

Eksileri ve Limitasyonları: Sistemin öngörülen en belirgin eksikliği, fiziki bariyer rölelerinin tam otomatik olarak açılması (IoT donanım katmanı) işleminin bu sürümde gerçekleştirilememiş olmasıdır.

Mevcut güvenlik politikaları ve kurum içi donanım tedarik kısıtları nedeniyle; sistem yetki doğrulamasını ve alarm yönetimini (Kırmızı/Yeşil uyarılar) saliseler içinde tam otonom yapsa da, geçiş onayından sonra fiziki bariyerin açılması şimdilik güvenlik görevlisinin manuel inisiyatifinde yürütülmektedir. Ayrıca sürekli açık kalan kamera ve yapay zeka analizi, mobil cihaz üzerinde batarya tüketimini hızlandırmaktadır.

10.2. Projenin Geliştiriciye Akademik ve Mesleki Katkıları

Bu projenin analiz, tasarım, gerçekleştirim ve sınama aşamalarının baştan sona tek bir çatı altında "Tam Yığın Geliştirici (Full-Stack Developer)" yaklaşımıyla yürütülmesi; teorik yazılım mühendisliği konseptlerinin gerçek dünya problemlerine uygulanması açısından eşsiz bir kazanım sağlamıştır. Geliştirme sürecinin sağladığı temel akademik ve teknik yetkinlikler şunlardır:

- **Mobil Uygulama ve Donanım Yönetimi:** Java programlama dili ile Android yaşam döngüsü (Lifecycle) derinlemesine kavranmış; cihaz kamerasından yüksek çözünürlüklü görüntü akışı alabilmek için Jetpack CameraX kütüphanesi başarıyla entegre edilmiştir. Ayrıca donanımsal titreşim (Vibrator) ve ses (ToneGenerator) motorlarının yazılımla nasıl senkronize edileceği pratik edilmiştir.
- **Yapay Zeka ve OCR Kavramları:** Google ML Kit kullanılarak makine öğrenmesi modellerinin (Makine Görüntüsü / Computer Vision) mobil platformlarda çevrimdışı (offline) olarak nasıl koşturulacağı tecrübe edilmiştir.

  Bilgisayarın anlamsız metin bloklarından doğru plakayı ayıklayabilmesi için Düzenli İfadeler (Regex) tabanlı algoritmalar tasarlanarak problem çözme yetisi geliştirilmiştir.

- **Dağıtık Mimari ve Asenkron Ağ İletişimi:** Python Flask çatısı kullanılarak RESTful API uç noktalarının nasıl tasarlanacağı öğrenilmiş; asenkron ağ isteklerinin Android Volley kütüphanesi ile nasıl yönetileceği ve Base64 ile şifrelenmiş görsel verilerin JSON formatında nasıl kayıpsız taşınacağı tecrübe edilmiştir.
- **Siber Güvenlik ve Veri Tabanı Mimarisi:** SQLite ile ilişkisel veritabanı kurma, primary key ve foreign key atamaları pratik edilmiştir. Personel parolalarının veritabanında açık metin (plain text) olarak saklanması yerine Werkzeug kütüphanesi kullanılarak kriptografik Hash'leme yöntemleriyle korunması gibi veri güvenliği standartları benimsenmiştir.
- **Yazılım Yaşam Döngüsü Uygulamaları:** İşlev Noktası ve COCOMO analizi yapılarak bir yazılımın maliyet ve kaynak kodu (LOC) kestirimlerinin nasıl hesaplanacağı öğrenilmiş, tasarımdan teste kadar yazılım mühendisliğinin teorik şablonları sahada birebir karşılık bulmuştur.

11.KAYNAKLAR

[1] Android Open Source Project. (2026). *CameraX Architecture and API Guide*. Geliştirici Dokümantasyonu.

Alındığı Yer: https://developer.android.com/training/camerax/architecture

[2] Google Developers. (2024). *ML Kit Vision API: Text Recognition v2*.

Alındığı Yer: https://developers.google.com/ml-kit/vision/text-recognition/v2

[3] Pallets Projects. (2026). *Flask: Web Development, One Drop at a Time*. Resmi API Dokümantasyonu.

Alındığı Yer: https://flask.palletsprojects.com/

[4] Android Open Source Project. (2026). *Transmitting Network Data Using Volley*.

Alındığı Yer: https://developer.android.com/training/volley

[5] SQLite Consortium. (2026). *SQLite Official Documentation and Architecture*.

Alındığı Yer: https://www.sqlite.org/docs.html

[6] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2. Baskı). O'Reilly Media.

[7] Mistry, R., & Misner, S. (2014). *Introducing Microsoft SQL Server 2014*. Microsoft Press. (YMT Tasarım evresi ve VTYS Şemaları teorik referansı).

[8] Sommerville, I. (2015). *Yazılım Mühendisliği (Software Engineering)* (10. Baskı). Pearson Education. (Helezonik Model, Sistem Çözümleme ve COCOMO analizi referansı).

[9] Pirim, A. G. H. (2006). *Yapay Zeka ve Makine Öğrenmesi Uygulamaları*. Yaşar Üniversitesi E-Dergisi, 1(1), 81-93.

[10] Google Samples GitHub Repository. (2024). ML Kit Android Quickstart (Vision ve OCR Entegrasyon Kodları).

Alındığı Yer: https://github.com/googlesamples/mlkit/tree/master/android/vision-quickstart

[11] Pallets Projects. (2024). Werkzeug Security Utilities (Kriptografik Şifreleme ve Şifre Hashleme).

Alındığı Yer: https://werkzeug.palletsprojects.com/en/3.0.x/utils/

Alındığı Yer: https://flask.palletsprojects.com/

[12] Ngrok Inc. (2024). Ngrok Secure Tunnels Official Documentation.

Alındığı Yer: https://ngrok.com/docs/

[13] Python Software Foundation. (2024). re--Regular expression operations (Düzenli İfadeler ile Plaka Filtreleme).

Alındığı Yer: https://docs.python.org/3/library/re.html

[14] Başer,U.K. (2021.12 Aralık). PYTHON İLE PLAKA TANIMA SİSTEMİ - (1) Plaka Tanıma Sistemi nedir ve öznitelik çıkartımı[Video].

Youtube: https://www.youtube.com/watch?v=H8Urx3MCok8

[15] U. K. Başer, "plakatanimasistemi," GitHub. [Çevrimiçi].

Erişim: https://github.com/umutkaanbaser/plakatanimasistemi